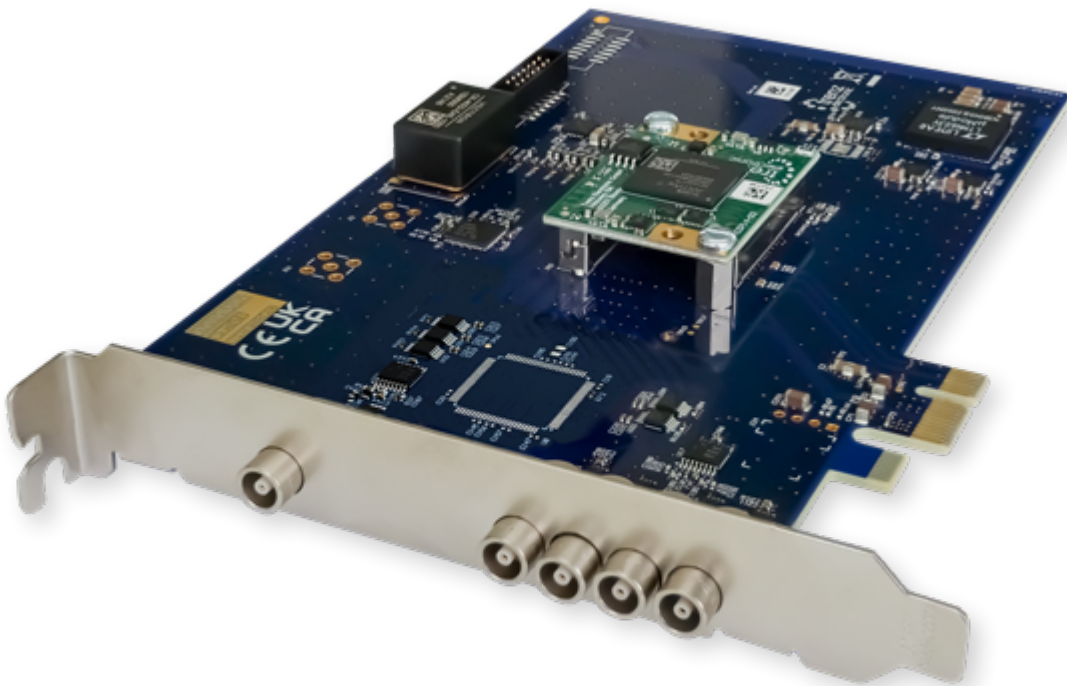


cronologic

TimeTagger4

User Guide



TimeTagger

Contents

Introduction	4
Features	5
TimeTagger4 Variants	6
Applications	6
1 Hardware	7
Installing the PCIe Board	7
Input and Connectors	7
Status LEDs of the PCIe boards	10
Memory Management	10
2 Functionality	11
Grouping and Events	11
Auto-Triggering Function Generator	12
Continuous Mode	12
Configurable Input Delay	13
Timing Generators (TiGer)	13
Performing a Firmware Update	15
Calibration of the 10G variant	16
3 API Reference	17
Constants and Types	18
General	18
Status Codes	18
cronologic Device IDs	19
Initialization	20
timetagger4_get_default_init_parameters	20
timetagger4_init	20
timetagger4_close	21
timetagger4_init_parameters	21
timetagger4_device	23
Configuration	24
timetagger4_configure	24
timetagger4_get_default_configuration	24
timetagger4_get_current_configuration	24
timetagger4_configuration	25
timetagger4_trigger	28
timetagger4_tiger_block	28
timetagger4_channel	30
timetagger4_delay_config	31
Runtime Control	32
timetagger4_start_capture	32
timetagger4_pause_capture	32
timetagger4_continue_capture	32
timetagger4_stop_capture	32
timetagger4_start_tiger	33
timetagger4_stop_tiger	33

Data Readout	34
timetagger4_read	34
timetagger4_acknowledge	35
crono_next_packet	35
timetagger4_read_in	36
timetagger4_read_out	36
Output Data Format	38
crono_packet	38
Status Information	42
timetagger4_get_device_type	42
timetagger4_get_last_error_message	42
timetagger4_get_fast_info	42
timetagger4_get_param_info	42
timetagger4_get_static_info	42
timetagger4_get_pcie_info	43
timetagger4_clear_pcie_errors	43
timetagger4_fast_info	43
timetagger4_param_info	44
timetagger4_static_info	45
crono_pcie_info	46
Driver Information	49
timetagger4_get_driver_revision	49
timetagger4_get_driver_revision_str	49
timetagger4_count_devices	49
4 Code Example	50
5 Ordering Information	58
6 Technical Data	59
Dimension	59
TDC Characteristics	59
TDC Measurement Characteristics for Gen 1	59
TDC Measurement Characteristics for Gen 2	60
Oscillator Time Base	61
Cable Delay Test	61
Electrical Characteristics	63
Power Supply	63
TDC Inputs	63
Information Required by DIN EN 61010-1	64
Manufacturer	64
Intended Use and System Integration	65
Environmental Conditions for Storage	65
Environmental Conditions for Operation	65
Cooling	65
Recycling	66
7 Revision History	67
Firmware	67
Gen 1	67
Gen 2	67
Driver & Applications	68
User Guide	69

Introduction

With the original TimeTaggers, cronologic has coined a name for an entire generic brand of time-to-digital converters. Our TimeTagger4 boards are *common-start*, low-resolution, high-throughput TDCs that offer the ability to record timestamps of leading or trailing edges (or both) of digital pulses.

The TimeTagger4 produces a stream of output *packets*, each containing data from a single start event.

The relative timestamps of all stop pulses that occur within a configurable time range are grouped into one packet, while all pulses outside that range are discarded.

This User Guide is also available online at docs.cronologic.de/timetagger4.

Features

- **4-channel common-start TDC**
- **Quantization** (measurement resolution): **100 to 1000 ps**
- **Standard range**: 8.388 ms for Gen 1 and **1.677 ms** for Gen 2 (24 bits)
- **Extended range**: 1.073 s for Gen 1 and **0.429 s** for Gen 2 (31 bits)
- **Double-pulse resolution**: twice the quantization size
- **Dead time** between groups: **none**
- **Minimum interval** between starts: 4 ns for Gen 1 and **3.2 ns** for **Gen 2**
- Up to **8000 Hits per packet**
- **5 to 0.625 GHz/s** for bursts of up to 4096 starts (depending on variant)
- **5 to 0.625 GHits/s/channel** for bursts of up to 3900 stops (depending on variant)
- **40 MHits/s/channel** of sustained stops
- **60 MHits/s** over all channels of sustained stops
- **PCIe x1** interface
- **Continuous mode**, supporting unlimited recording of stops without starts

TimeTagger4 Variants

The TimeTagger4 exists in multiple variants. Each variant exists as a PCIe board or as a standalone desktop solution. For more information see [Ordering Information](#).

Table 1 TimeTagger4 variants.

Version	-1G	-2G	-1.25G	-2.5G	-5G	-10G
Quantization	1000 ps	500 ps	800 ps	400 ps	200 ps	100 ps
Data Format Bin Size	500 ps	500 ps	100 ps	100 ps	100 ps	100 ps
DNL and INL	0.01 LSB	0.01 LSB	0.01 LSB	0.01 LSB	0.01 LSB	0.3 LSB
PCIe line rate	Gen 1	Gen 1	Gen 2	Gen 2	Gen 2	Gen 2
Readout Rate	200 MB/s	200 MB/s	400 MB/s	400 MB/s	400 MB/s	400 MB/s
Status	end of life	end of life	active	active	active	active

Applications

The TimeTagger4 can be used in all time measurement applications where a common-start setup with 100 ps resolution is sufficient.

For alternatives with higher resolution, more channels, or higher readout rates check our TDC website at www.cronologic.de.

The TimeTagger4 is well-suited for the following applications:

- LIDAR down to 8 cm resolution
- Blade oscillation measurements
- Reciprocal counters
- Coincidence measurements
- Quantum key distribution (QKD)
- Time-correlated single-photon counting (TCSPC)

1 Hardware

The TimeTagger4 is available as a PCIe plugin board (variant “-PCIe”) or as a desktop solution (variant “-TBT”). The two variants are shown in [Figure 1.1](#).



Figure 1.1 Overview of the PCIe (left) and the TBT (right) variant of the TimeTagger4.

Installing the PCIe Board

The TimeTagger4 board can be installed in any PCIe-CEM slot with x1 or more lanes. Make sure that the PC is powered off and the main power connector is disconnected while installing the board.

Input and Connectors

[Figure 1.2](#) shows the location on the inputs on the slot bracket. [Figure 1.3](#) shows a side view of the PCIe variant of the TimeTagger4.

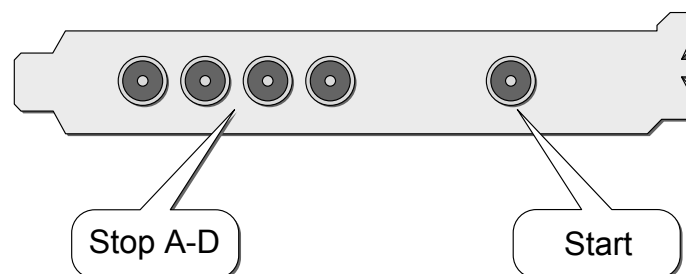


Figure 1.2 Input connectors of the TimeTagger4 on the PCIe bracket.

LEMO-00 connectors are used for input connection. The inputs are **AC-coupled** and have an impedance of 50 Ω . A schematic of the input circuit is shown in [Figure 1.4](#).

The digital threshold for each input can be adjusted individually to comply with a multitude of single-ended signaling standards (see [timetagger4_configuration.dc_offset](#)). The threshold can also be used to configure the input for either positive or negative pulses.

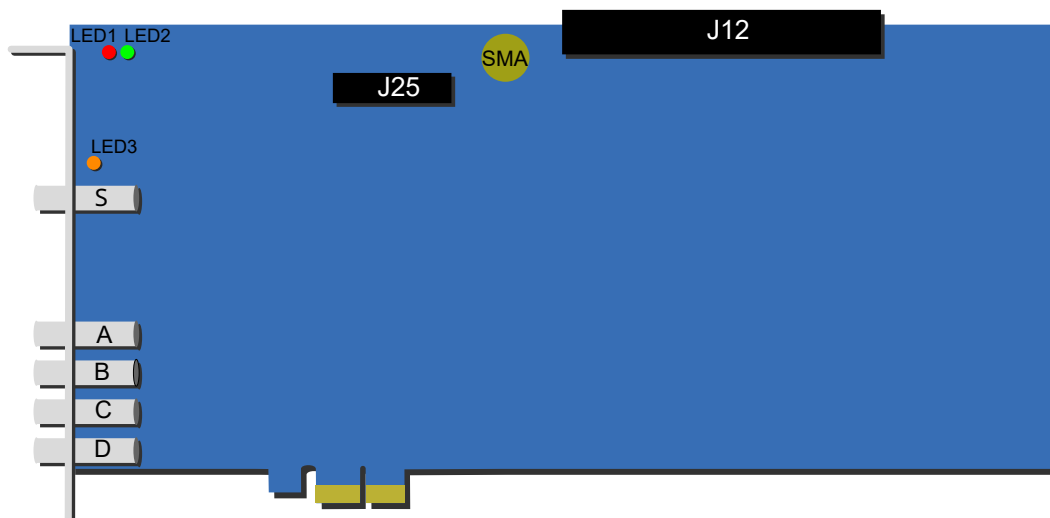


Figure 1.3 Schematic of the TimeTagger4 PCIe board. The connectors J12 and J25 are only present on Gen 1 boards.

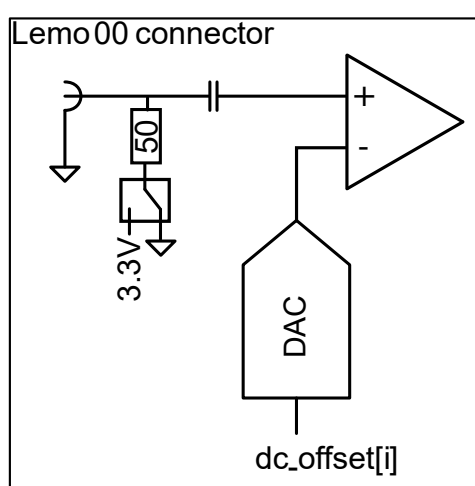


Figure 1.4 Input circuit of the TDC channels.

The connectors can also be used as outputs. DC-coupled output pulses for automatic internal triggering and control of external devices can be generated using the [Timing Generators \(TiGer\)](#).

The SMA connector at the top edge of the board can be used to connect an external 10 MHz clock signal. To enable use of an external clock, configure `timetagger4_init_parameters.use_ext_clock` accordingly.

Only Gen 1 TimeTagger4 boards have inter-board connectors near the top edge of the board. The pinout of connector J12 is shown in [Table 1.1](#). The connector J12 is reserved for future use.

Table 1.1 Pinout of connector J12.

Pin	Name
1, 2	GND
3, 4	external CLK in N, external CLK in P
5, 6	GND
7, 8	reserved/NC
9, 10	GND
11, 12	reserved/NC
13, 14	GND
15, 16	reserved/NC
17, 18	GND
19, 20	reserved/NC
21, 22	GND
23, 24	reserved/NC
25, 26	GND
27, 28	reserved/NC
29, 30	GND
31, 32	reserved/NC
33, 34	GND

Status LEDs of the PCIe boards

Note

This section refers to the status LEDs of the PCIe board, not the LEDs of the TBT variant (see [here](#)).

Three status LEDs are present on the board, as seen in [Figure 1.3](#).

- LED1 lights up red during the configuration of the FPGA and turns off afterward. If it stays permanently lit, the configuration failed.
- LED2 lights up green after the board is initialized by the driver and turns off when the device is closed by the software.
- LED3 lights up green when capture is started, yellow after the first start signal was detected and red when groups are missing.

Memory Management

The TimeTagger4 has internal FIFOs (first-in, first-out) that buffer data during acquisition.

The data is streamed from the FIFO to the host PC and stored in the *host buffer*. Data will only be overwritten in the host buffer if it has been *acknowledged*.

The host buffer is managed by the DMA (direct memory access) driver. The DMA driver can only ever write to the host buffer if enough memory is free. That means, new packets will never overwrite old packets unless they have been acknowledged.

If the host buffer is full, data may be lost. If this occurred, the corresponding packets will have the `TIMETAGGER4_PACKET_FLAG_HOST_BUFFER_FULL` bit of `crono_packet.flags` will be set. This *may* result in lost packets.

If the hit rate is too high, the internal FIFOs may fill up. If this is the case, the affected packets will have the `TIMETAGGER4_PACKET_FLAG_DMA_FIFO_FULL` bit of `crono_packet.flags` will be set. This *may* result in lost packets. However, only if the `TIMETAGGER4_PACKET_FLAG_SHORTENED` bit of `crono_packet.flags` is set, packets were actually missed.

2 Functionality

The TimeTagger4 is a “classic” common-start time-to-digital converter.

It records the time difference between a leading or trailing edge on the start input to the leading or trailing edges of the stop inputs (called **timestamps**).

Detection of rising and falling edges of the stop channels A–D can be enabled individually.

The time measurements are quantized. The quantization depends on the specific variant of the TimeTagger4, ranging from **100 to 1000 ps** (see [Table 1](#)).

The timestamps are recorded in integer multiples of the data bin size (500 ps for Gen 1 and **100 ps** for Gen 2).

Throughout this User Guide, transitions of the input signals are called **hits**.

To reliably detect hits, the signal has to be stable for more than one quantization interval before and after the edge.

Triggers on the Start channel must not occur less than **5 ns** apart.

The TimeTagger4 records events without dead time at a readout rate of about 48 MHits/s for Gen 1 and **60 MHits/s** for Gen 2. For Gen 2, the maximum readout rate of a single channel is **40 MHits/s**.

Grouping and Events

In typical applications, a start hit is followed by a multitude of stop hits.

The TimeTagger4 manages hits in groups (in some applications, a group of hits is often called an “event”).

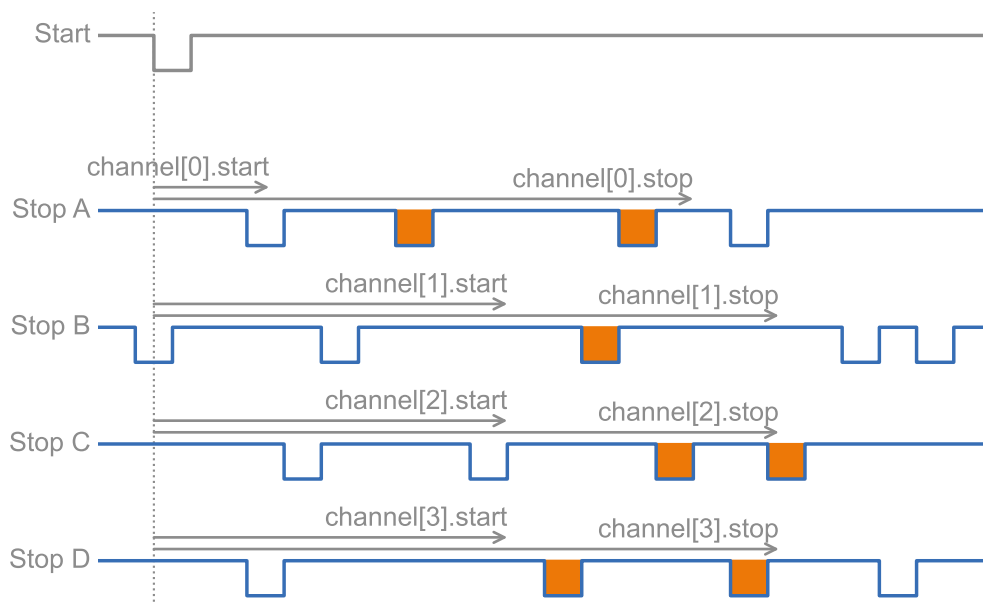


Figure 2.1 Principle of the grouping functionality. Only the orange hits where the leading edge falls in-between *start* and *stop* are grouped, others discarded. Alternatively, trailing edges or both edges may be used to determine if a hit should be kept.

[Figure 2.1](#) shows a corresponding timing diagram. For each Stop channel, you can define the range

within which events are grouped. Hits outside that range are discarded.

Each Stop channel is configured by setting the *start* and *stop* values of the corresponding *channel[i]*.

If a second start hit is recorded within the range of a group, the current group is finished and a new group is started. Consecutive stop hits will be assigned to the new group (as long as they are within the group range).

For Gen 1, the maximum group range is $0 \leq \text{start} \leq \text{stop} \leq 2^{31}$.

For Gen 2, the **maximum group range** is $0 \leq \text{start} \leq \text{stop} \leq 2^{32}$.

Start and *stop* are in units of *timetagger4_param_info.binsize* (which is 500 ps for Gen 1 and 100 ps for Gen 2). Ergo, the maximum group range is 1.073 s for Gen 1 and 0.429 s for Gen 2.

Note

For group ranges $\text{stop} - \text{start} > 2^{24}$, the timestamps will contain *rollover* (see *TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW*.)

Auto-Triggering Function Generator

Some applications require internal periodic or random triggering. The TimeTagger4 auto trigger function generator provides this functionality.

The delay between two trigger pulses of this trigger generator is the sum of two components: A fixed value *M* and a pseudo-random value with a range given by the exponent *N*.

The period is

$$T = M + [1 \dots 2^N] - 1$$

clock cycles with a duration of 4 ns per cycle for Gen 1 and 3.2 ns for Gen 2.

The standard values of *M* = 62500 and *N* = 0 result in a frequency of 4 kHz for TimeTagger4 Gen 1 and 5 kHz for Gen 2.

M and *N* are configured with *timetagger4_configuration.auto_trigger_period* and *timetagger4_configuration.auto_trigger_random_exponent*, respectively.

The trigger can be used as a source for the TiGer unit [see *Timing Generators (TiGer)*], which enables internal triggering of the Start channel.

The trigger defines the period for the continuous mode (see *Continuous Mode*).

Continuous Mode

This feature is only available for TimeTagger4 Gen 2.

In continuous mode, the TimeTagger4 continuously records stop signals even without a start signal.

The data stream contains periodic packets with an absolute 64-bit timestamp, followed by a list of stop timestamps relative to the absolute one.

The frequency of absolute timestamps can be adjusted using the *Auto-Triggering Function Generator*.

Lower frequencies will create larger packets and have a larger latency for receiving packets. This potentially overflows the *buffers*.

Frequencies lower or equal to 600 Hz will contain rollover (see *TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW*).

Disregarding these points, the choice of frequency is arbitrary.

⚠ Attention

For continuous mode, *timetagger4_channel.start* and *timetagger4_channel.stop* should be kept at their default values (as provided by *timetagger4_get_default_configuration()*).

Configurable Input Delay

This feature is only available for TimeTagger4 Gen 2.

Each of the five input channels of the TimeTagger4 can be delayed for up to 204.6 ns with a 200 ps granularity.

Timing Generators (TiGer)

Each digital LEMO-00 input can be used as an LVCMOS trigger output.

The TiGer functionality can be configured for each connector independently. See *timetagger4_tiger_block* for a full description of all configuration options.

Figure 2.2 shows how the TiGer blocks are connected. They can be triggered by an OR of an arbitrary combination of inputs, including the auto trigger. Each TiGer can drive its output to its corresponding LEMO connector. This turns the connector into an output.

The TiGer is DC coupled to the connector. Connected hardware must not drive any signals to connectors that are used as outputs, as doing so could damage both the TimeTagger4 and the external hardware. Pulses that are short enough for the input AC coupling are available as input signals to the TimeTagger4. This can be used to measure exact time differences between the generated output signals and input signals on other channels.

When using one of the input channels as a source for the TiGer, the expected latency between signal input and TiGer output is roughly 95 ns.

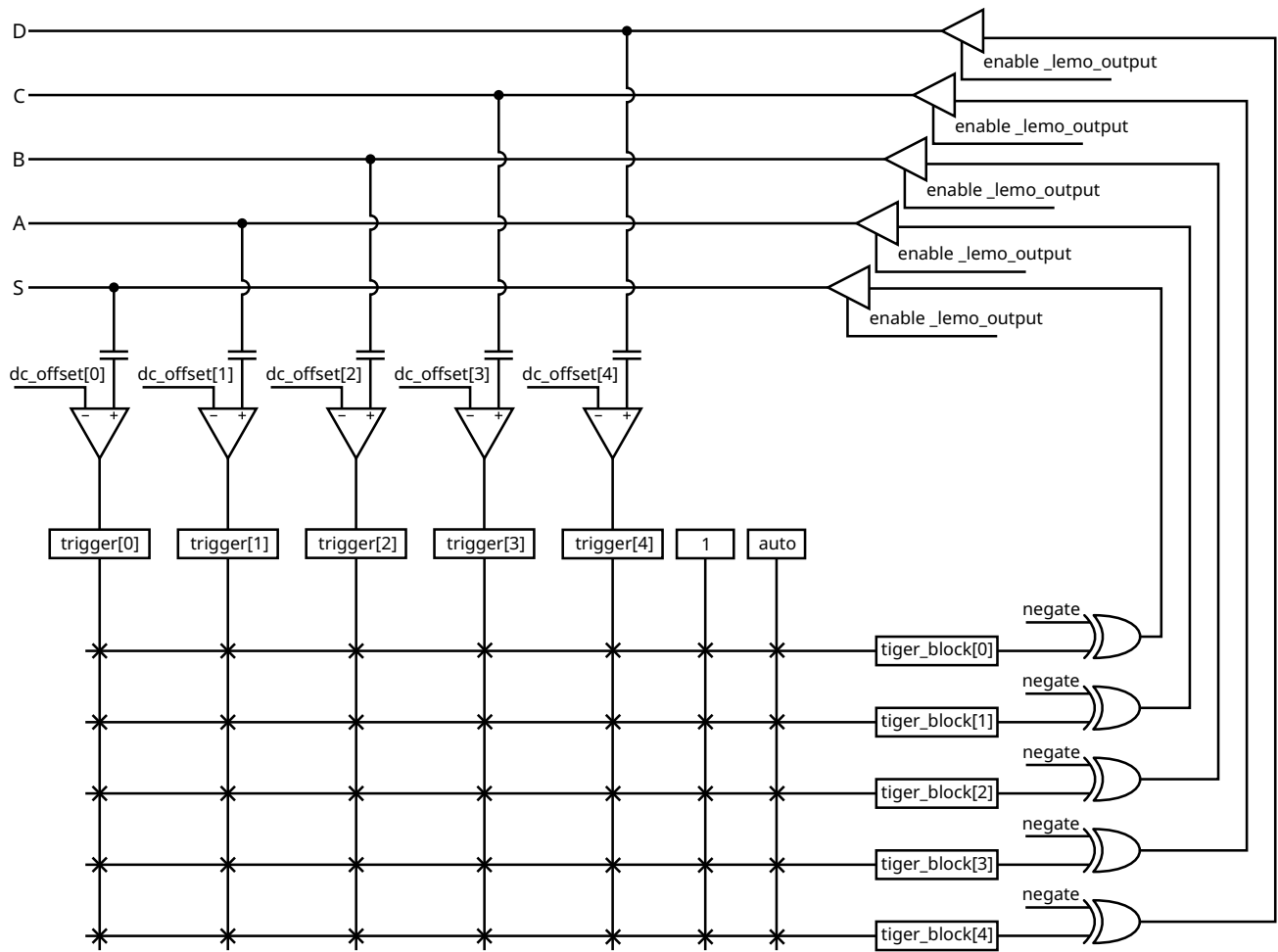


Figure 2.2 Routing logic of the TiGer blocks. The LEMO inputs can trigger the TiGer blocks, which can enable outputs on the LEMO connectors.

Performing a Firmware Update

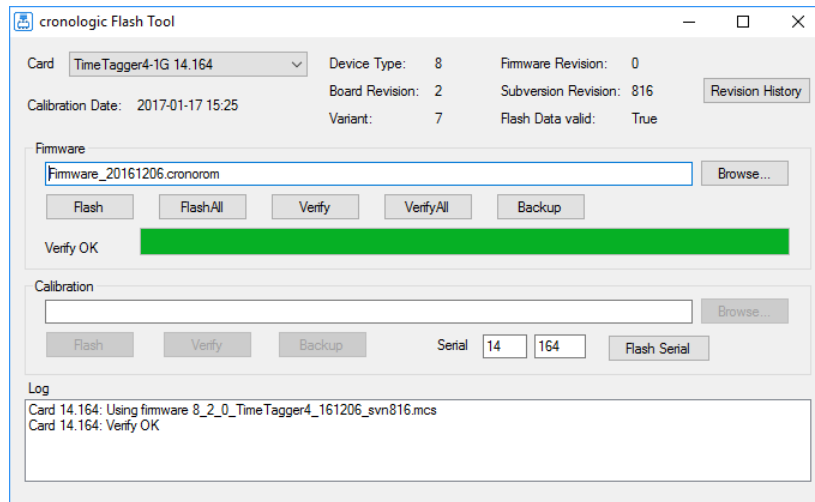


Figure 2.3 Screenshot of the TimeTagger4's FirmwareGUI.

After installing the TimeTagger4 device driver, a firmware update tool is available. By choosing *FirmwareGUI.exe* a firmware update can be performed (the default location is under `C:\Program Files\cronologic\TimeTagger4\apps\x64\`) After invoking the application, a window as shown in [Figure 2.3](#) will appear. The tool can be used for updating the firmware and to create a backup of the on-board calibration data of the TimeTagger4 unit.

If several boards are present, the one which is going to be used can be selected in the upper left corner of the window.

When pressing one of the *Backup* buttons, a backup of the firmware or the calibration data will be created, respectively.

In order to perform a firmware update, chose the `.cronorom`-file to be used by pressing *Browse*. The file contains the firmware data. By pressing *Flash*, the firmware is written to the board.

Verify can be used to compare the firmware data stored on the TimeTagger4 to the one provided by a file.

Flash All and *Verify All* perform the corresponding operation on all boards which are installed.

Attention

The new firmware will only be used by the board after a power cycle, i.e. after switching the PC (or Ndigo crate) off and back on.

A simple reboot is not sufficient. Therefore, the information shown in the upper half of the application window does not change right after flashing a new firmware.

Note

After a firmware update, the TimeTagger4-10G (and only the 10G variant) must be re-calibrated.

Calibration of the 10G variant

After performing a firmware update, the TimeTagger-10G variant has to be re-calibrated. This does *only* apply to the 10G variant.

For this purpose, cronologic provides the `timetagger4_10g_calibration` command-line tool. The tool is located in the driver installation directory in `apps\x64` (by default `C:\Program Files\cronologic\TimeTagger4\apps\x64\timetagger4_10g_calibration_64.exe`).

To perform a calibration, you need a NIM signal with a constant frequency larger than 20 kHz and smaller than 15 MHz. The stability of the frequency is crucial. A period stability of 6 ps has been verified to work.

Run `timetagger4_10g_calibration_64.exe` and follow the instructions on-screen, that is:

- Connect the NIM signal to the Start channel and press enter. The tool will calibrate the start channel.
- After Start was successfully calibrated, connect the same NIM signal to channel A and press enter. The tool will calibrate channel A.
- Repeat the last step for channels B, C, and D.

After a successful calibration of the TimeTagger4-10G, the message “Calibrated all channels successfully” will be displayed. You can close the calibration tool.

In case the calibration fails, please check the following:

- The TimeTagger4-10G is installed properly (see [Installing the PCIe Board](#)).
- A proper NIM signal with a constant frequency larger than 20 kHz and smaller than 10 MHz is used.
- The NIM signal used conforms to the signal requirements laid out in [TDC Inputs](#).
- The NIM signal was connected to the appropriate input channel (see [Figure 1.2](#)).

3 API Reference

The API is a DLL with C linkage.

The functions provided by the driver are declared in `timeTagger4_interface.h` which must be included by your source code. You must tell your compiler to link with the file `xTDC4_driver_64.lib`.

When running your program the dynamic link library containing the actual driver code must reside in the same directory as your executable or be in a directory included in the PATH variable. For Linux, it is provided only as a static library `libxtdc4_driver.a`. The file for the DLL is called `xTDC4_driver_64.dll`. All these files are provided with the driver installer that can be downloaded from the product website at www.cronologic.de.

By default, the installer will place the files into the directory `C:\Program Files\cronologic\TimeTagger4\driver`.

Setting up and using a TimeTagger4

Setting up and using a TimeTagger4 in your application software requires that you:

- Initialize a TimeTagger4 device (see [Initialization](#)).
- Configure the initialized TimeTagger4 device (see [Configuration](#)).
- Start data acquisition (see [Runtime Control](#)).
- Read data from the device (see [Data Readout](#)).
- Process the data provided as packets (see [Output Data Format](#)).

Constants and Types

General

TIMETAGGER4_TDC_CHANNEL_COUNT

The number of TDC input channels.

TIMETAGGER4_TIGER_COUNT

The number of *timing generators*.

TIMETAGGER4_TRIGGER_COUNT

The number of potential trigger sources for the *timing generators*.

One for each TDC input and one for the Start input, as well as some special trigger. See [timetagger4_tiger_block.sources](#) for more information.

TIMETAGGER4_API_VERSION

Current version of the API.

type crono_bool_t

Data type used for booleans in various structs.

Status Codes

TIMETAGGER4_OK

0. Return value of various API functions if no error occurred.

TIMETAGGER4_WRONG_STATE

4. The TimeTagger4 is in the wrong state for the requested action (e.g., continuing to capture when the capture was not paused).

TIMETAGGER4_INVALID_DEVICE

5. The device pointer passed to various API functions was invalid.

TIMETAGGER4_INVALID_BUFFER_PARAMETERS

8. Invalid buffer parameters were used when configuring a TimeTagger4 device.

TIMETAGGER4_INVALID_CONFIG_PARAMETERS

9. The parameters used for configuring a TimeTagger4 device were invalid.

TIMETAGGER4_HARDWARE_FAILURE

11. A hardware failure occurred.

TIMETAGGER4_DEVICE_OPEN_FAILED

14. Failed to open the TimeTagger4 device during initialization.

TIMETAGGER4_CRONO_INTERNAL_ERROR

15. An internal error occurred.

TIMETAGGER4_CRONO_INVALID_ARGUMENTS

17. The arguments passed to various API functions were invalid.

cronologic Device IDs

CRONO_DEVICE_TIMETAGGER4

Initialization

To initialize a TimeTagger4:

- Get a default set of initialization parameters using `timetagger4_get_default_init_parameters()`.
- Change parameters of `timetagger4_init_parameters` according to your specific requirements.
- Initialize a TimeTagger4 device using `timetagger4_init()`.
- Release all resources using `timetagger4_close()`.

The following example shows a basic setup of a TimeTagger4 using the default initialization parameters. A complete coding example can be found on github.com/cronologic-de/xtdc_babel or in Section [Code Example](#).

```
// get a default set of initialization parameters
timetagger4_init_parameters params;
status = timetagger4_get_default_init_parameters(&params);
if (status != TIMETAGGER4_OK) { /* handle error */ }

// initialize a device
const char* err_message;
timetagger4_device* device = timetagger4_init(&params, &status, &err_message);
if (status != TIMETAGGER4_OK) { /* handle error */ }

// use device

// after usage, free up all resources by closing the device
status = timetagger4_close(device)
```

int timetagger4_get_default_init_parameters(*timetagger4_init_parameters* *init)

Initialize a `timetagger4_init_parameters` struct with default values.

You should always use this method first to set up your initialization parameters, then adjust the parameters to your specific needs.

Parameters

- **init** – Pointer to a `timetagger4_init_parameters` struct that will be filled.

Returns

Status codes: `TIMETAGGER4_OK` or `TIMETAGGER4_CRONO_INVALID_ARGUMENTS`.

***timetagger4_device* timetagger4_init(*timetagger4_init_parameters* *params, int *error_code, char **error_message)**

Opens and initializes the TimeTagger4 board with index `params.card_index`.

Parameters

- **params** – Pointer to a `timetagger4_init_parameters`. The struct must have been completely initialized using `timetagger4_get_default_init_parameters()`.
- **error_code** – Pointer to the location where a potential error code will be stored. Equals `TIMETAGGER4_OK` if no error occurred. Otherwise can be `TIMETAGGER4_CRONO_INVALID_ARGUMENTS`,

TIMETAGGER4_DEVICE_OPEN_FAILED, *TIMETAGGER4_HARDWARE_FAILURE*, *TIMETAGGER4_INVALID_BUFFER_PARAMETERS*, *TIMETAGGER4_CRONO_INTERNAL_ERROR*, or *TIMETAGGER4_WRONG_STATE*.

- **error_message** – Pointer to a location where a potential error message in plain text will be stored.

Returns

The TimeTagger4 device corresponding to *params.card_index*.

int timetagger4_close(*timetagger4_device* *device)

Close an initialized TimeTagger4 device, releasing all resources.

Parameters

- **device** – Pointer to the device that shall be closed.

Returns

Status code: *TIMETAGGER4_OK*, *TIMETAGGER4_INVALID_DEVICE*, *TIMETAGGER4_WRONG_STATE*, or *TIMETAGGER4_CRONO_INTERNAL_ERROR*.

struct timetagger4_init_parameters

Struct for the initialization of the TimeTagger4.

This structure must be completely initialized. For this, obtain a default set of initialization parameters first by using *timetagger4_get_default_init_parameters()*, then adjust parameters for your use case.

int version

The version number.

Must be set to *TIMETAGGER4_API_VERSION*.

int card_index

The index in the list of TimeTagger4 boards that should be initialized.

There might be multiple boards in the system that are handled by the driver as reported by *timetagger4_count_devices()*. This index selects one of them.

Boards are enumerated depending on the PCIe slot. The lower the bus number and the lower the slot number the lower the card index.

int board_id

The global index in all cronologic devices.

This 8-bit number is filled into each packet created by the board and is useful if data streams of multiple boards are merged.

If only TimeTagger4 cards are used this number can be set to the *card_index*.

If boards of different types that use a compatible data format are used in a system, each board should get a unique ID.

uint64_t buffer_size[8]

The minimum size of the DMA buffer in bytes.

Defaults to $4 \times 1024 \times 1024$ bytes.

The minimal possible size is 1024×1024 bytes.

For the TimeTagger4, only the first entry is used.

int `buffer_type`

The type of buffer.

Must be one of the following:

`TIMETAGGER4_BUFFER_ALLOCATE`

Use allocated buffer.

`TIMETAGGER4_BUFFER_USE_PHYSICAL`

Use physical buffer (currently not supported).

uint64_t `buffer_address`

The start address of the reserved memory.

The buffers will be allocated with the sizes given above. Make sure that the memory is large enough.

int `variant`

A variant, for reconfiguring the chip for future extension. Currently fixed at 0.

int `device_type`

A constant for the different devices from cronologic.

Initialized by `timetagger4_get_default_init_parameters()` to `CRONO_DEVICE_TIMETAGGER4`.

Must be left unchanged.

int `dma_read_delay`

The update delay of the writing pointer after a packet has been send over PCIe.

Given in multiples of 16 ns.

Must be left unchanged.

int `use_ext_clock`

Select external 10 MHz reference.

If set to 1 use external 10 MHz reference.

If set to 0 use internal reference.

int `rclk_sel`

Set THS788 RClk frequency, default is `TDC4_RCLK_150M`.

Must be one of the following:

`TDC4_RCLK_37M5`

37.5 MHz

`TDC4_RCLK_75M`

75 MHz

`TDC4_RCLK_150M`

150 MHz

struct **timetagger4_device**

Structure referencing an initialized TimeTagger4 device.

void ***timetagger4**

Configuration

After a TimeTagger4 device has been initialized (see [Initialization](#)), it must be configured before it can acquire data.

To configure a TimeTagger4 device:

- Get a default set of configuration parameters using `timetagger4_get_default_configuration()`.
- Change parameters of `timetagger4_configuration` according to your specific requirements.
- Configure the TimeTagger4 device using `timetagger4_configure()`.

The following example shows a basic configuration of an already initialized TimeTagger4 device using the default configuration parameters. A complete coding example can be found on github.com/cronologic-de/xtdc_babel or in Section [Code Example](#).

```
// get a default set of configuration parameters
timetagger4_configuration config;
status = timetagger4_get_default_configuration(device, &config);
if (status != TIMETAGGER4_OK) { /* handle error */ }

// configure the device
status = timetagger4_configure(device, &config);
if (status != TIMETAGGER4_OK) { /* handle error */ }
```

`int timetagger4_configure(timetagger4\_device *device, timetagger4\_configuration *config)`

Configures a TimeTagger4 device.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **config** – Pointer to a `timetagger4_configuration` struct used for the configuration.

Returns

Status code: `TIMETAGGER4_OK`, `TIMETAGGER4_INVALID_DEVICE`, `TIMETAGGER4_HARDWARE_FAILURE`, or `TIMETAGGER4_INVALID_CONFIG_PARAMETERS`.

`int timetagger4_get_default_configuration(timetagger4\_device *device, timetagger4\_configuration *config)`

Obtain a default set of configuration parameters.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **config** – Pointer to a `timetagger4_configuration` struct that will be filled.

Returns

Status code: `TIMETAGGER4_OK`, `TIMETAGGER4_INVALID_DEVICE`, or `TIMETAGGER4_CRONO_INVALID_ARGUMENTS`.

`int timetagger4_get_current_configuration(timetagger4\_device *device, timetagger4\_configuration *config)`

Obtain the current set of configuration parameters of an already configured TimeTagger4 device.

Parameters

- **device** – Pointer to a configured TimeTagger4 device.
- **config** – Pointer to a `timetagger4_configuration` struct that will be filled.

Returns

Status code: `TIMETAGGER4_OK`, or `TIMETAGGER4_INVALID_DEVICE`.

struct **timetagger4_configuration**

This struct is used to configure a TimeTagger4 device using `timetagger4_configure()`.

int **size**

The number of bytes this struct occupies.

int **version**

Version number of this struct. It is increased when the definition of the struct changes.

int **tdc_mode**

Operational mode of the TDC.

Must be one of the following:

TIMETAGGER4_TDC_MODE_GROUPED

Operate in grouped mode.

Classical Common-Start operation. See also [Grouping and Events](#).

TIMETAGGER4_TDC_MODE_CONTINUOUS

Operate in continuous mode. Only available for Gen 2.

See also [Continuous Mode](#).

`auto_trigger_period` must be set appropriately.

Attention

The time corresponding to `channel[i].stop` must be larger than the time corresponding to `auto_trigger_period`, otherwise not all hits on `channels[i]` will be recorded.

You can achieve this easily by keeping `stop` at its the default value `0x7fffffff` (the maximum possible value).

`crono_bool_t start_rising`

Not applicable for the TimeTagger4. Rising and/or falling edges are configured using `timetagger4_trigger`.

double **dc_offset**[`TIMETAGGER4_TDC_CHANNEL_COUNT` + 1]

Set the threshold voltage for the input channels S, A–D.

See also [Figure 1.4](#).

- `dc_offset[0]`: Threshold for channel Start.
- `dc_offset[1–4]`: Thresholds for Stop channels A–D.

The supported range is –1.27 to 1.13 V.

The threshold should be close to 50% of the height of the input pulse.

The effective resolution is about ± 4 mV.

Note

The inputs are AC coupled. Thus, the absolute voltage is not important for pulse inputs. It is the relative pulse amplitude that causes the input circuit to switch. The parameter must be set to the relative switching voltage for the input standard in use.

If the pulses are negative, a negative switching threshold must be set and vice versa.

Attention

The supported range changed for [driver release 1.10.7](#). That means, if you use a value for `dc_offset` outside the new supported range in your source code, the device configuration will adjust it automatically to the new supported range (e.g., a value of 1.18 V will be reduced to 1.13 V).

Values for various signaling standards are provided as macros:

TIMETAGGER4_DC_OFFSET_P_NIM

DC offset is set to 0.35 V.

TIMETAGGER4_DC_OFFSET_P_CMOS

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_LVCMOS_33

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_LVCMOS_25

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_LVCMOS_18

DC offset is set to 0.90 V.

TIMETAGGER4_DC_OFFSET_P_TTL

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_LVTTL_33

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_LVTTL_25

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_SSTL_3

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_P_SSTL_2

DC offset is set to 1.13 V.

TIMETAGGER4_DC_OFFSET_N_NIM

DC offset is set to –0.35 V.

TIMETAGGER4_DC_OFFSET_N_CMOS

DC offset is set to -1.27 V.

TIMETAGGER4_DC_OFFSET_N_LVCMOS_33

DC offset is set to -1.27 V.

TIMETAGGER4_DC_OFFSET_N_LVCMOS_25

DC offset is set to -1.25 V.

TIMETAGGER4_DC_OFFSET_N_LVCMOS_18

DC offset is set to -0.90 V.

TIMETAGGER4_DC_OFFSET_N_TTL

DC offset is set to -1.27 V.

TIMETAGGER4_DC_OFFSET_N_LVTTL_33

DC offset is set to -1.27 V.

TIMETAGGER4_DC_OFFSET_N_LVTTL_25

DC offset is set to -1.25 V.

TIMETAGGER4_DC_OFFSET_N_SSTL_3

DC offset is set to -1.27 V.

TIMETAGGER4_DC_OFFSET_N_SSTL_2

DC offset is set to -1.25 V.

***timetagger4_trigger* trigger**[TIMETAGGER4_TRIGGER_COUNT]

Configuration of the polarity of the external trigger sources.

External trigger sources are used as inputs for the TiGer blocks and as inputs to the time measurement unit.

Index 0 refers to the Start channel, indices 1 through 4 to the Stop channels A through D.

***timetagger4_tiger_block* tiger_block**[TIMETAGGER4_TIGER_COUNT]

Configuration of the *Timing Generators*.

Index 0 refers to the TiGer connected to the Start channel, indices 1 through 4 to the TiGer-Units connected to the Stop channels A through D.

***timetagger4_channel* channel**[TIMETAGGER4_TDC_CHANNEL_COUNT]

Configuration of the Stop channels.

Indices 0 through 3 refer to the Stop channels A through D.

timetagger4_lowres_channel lowres_channel[TIMETAGGER4_LOWRES_CHANNEL_COUNT]

Not applicable for the TimeTagger4.

This field is part of the struct to ensure driver compatibility with other cronologic boards.

uint32_t auto_trigger_period

Configure the base frequency of the *auto trigger function generator*.

See also *auto_trigger_random_exponent*.

uint32_t auto_trigger_random_exponent

Configure the randomness of the *auto trigger function generator*.

There is no enable or reset of the auto trigger.

Given the two parameters *auto_trigger_period* (M) and *auto_trigger_random_exponent* (N), the frequency T of the auto trigger function generator will be

$$T = M + [1 \dots 2^N] - 1$$

Depending on *tdc_mode*, the following restrictions apply

TIMETAGGER4_TDC_MODE_GROUPED

$$M_{\min} \leq M < 2^{32} \text{ and } 0 \leq N < 32$$

$M_{\min}$ is 6 for Gen 1 and 8 for Gen 2.

TIMETAGGER4_TDC_MODE_CONTINUOUS

$$31 \leq M < 78125000 \text{ and } 0 \leq N < 32$$

M and N are given in units of *timetagger4_static_info.auto_trigger_ref_clock*.

Note

The auto trigger can be used as a source of the TiGer blocks (*timetagger4_tiger_block.sources*).

timetagger4_delay_config **delay_config**[*TIMETAGGER4_TDC_CHANNEL_COUNT* + 1]

Configuration of the channel delays.

Index 0 refers to the Start channel, indices 1 through 4 to the Stop channels A through D.

uint32_t ignore_empty_packets

If enabled (any value but 0), do not write empty packets to the output stream.

Disabled by default.

struct timetagger4_trigger

Configure if rising or falling or both edges create a trigger event for *timetagger4_tiger_block*.

crono_bool_t **falling**

Falling edges will trigger an event.

crono_bool_t **rising**

Rising edges will trigger an event.

struct timetagger4_tiger_block

This struct configures the Timing Generators [see also *Timing Generators (TiGer)*].

crono_bool_t **enable**

Activates the TiGer.

Note

To use a TiGer, make sure to also set *enable_lemo_output* to true.

crono_bool_t **negate**

Inverts the output polarity.

Default is false.

`crono_bool_t retrigger`

Enables re-triggering.

If re-triggering is enabled, the following applies: After an event triggered the TiGer output, another event that is detected before the `stop` value is reached will extend the TiGer output for a time `stop` minus `start`.

`crono_bool_t extend`

Not applicable for the TimeTagger4.

`crono_bool_t enable_lemo_output`

Enables the LEMO output.

Drive the TiGer signal to the corresponding LEMO connector as an output.

Pulses created by the TiGer are visible at the corresponding input and can be used to get their exact timing.

Attention

The output is DC coupled. Make sure to not connect any devices to the corresponding input, as this may damage the device or the TimeTagger4.

`uint_32_t start`

Configure the duration when the TiGer output is enabled relative to the trigger.

See also `stop`.

`uint_32_t stop`

Configure the duration when the TiGer output is enabled relative to the trigger.

In multiplies of 4 ns for Gen 1 and 3.2 ns for Gen 2.

The range is $0 \leq \text{start} \leq \text{stop} \leq 2^{16} - 1$.

After a trigger event, a timer starts. Once the timer reaches `start`, the TiGer activates. Once the timer reaches `stop`, the TiGer deactivates.

If another trigger event is detected before the timer reaches `stop` and `retrigger` is true, the timer resets to `start`.

`int sources`

A bitmask with a bit set for all trigger sources that can trigger this TiGer block.

Default is `TIMETAGGER4_TRIGGER_SOURCE_S`.

Possible trigger sources are:

`TIMETAGGER4_TRIGGER_SOURCE_S`

0x00000001. If this bit is set, the Start channel can trigger the TiGer block.

`TIMETAGGER4_TRIGGER_SOURCE_A`

0x00000002. If this bit is set, Stop channel A can trigger the TiGer block.

`TIMETAGGER4_TRIGGER_SOURCE_B`

0x00000004. If this bit is set, Stop channel B can trigger the TiGer block.

TIMETAGGER4_TRIGGER_SOURCE_C

0x00000008. If this bit is set, Stop channel C can trigger the TiGer block.

TIMETAGGER4_TRIGGER_SOURCE_D

0x00000010. If this bit is set, Stop channel D can trigger the TiGer block.

TIMETAGGER4_TRIGGER_SOURCE_AUTO

0x00004000. If this bit is set, the *auto trigger function generator* can trigger the TiGer block.

TIMETAGGER4_TRIGGER_SOURCE_ONE

0x00008000. If this bit is set, the TiGer block is triggered every clock cycle.

For example, if you want the Start channel and the auto trigger to trigger the TiGer block:

```
int bitmask = TIMETAGGER4_TRIGGER_SOURCE_S | TIMETAGGER4_TRIGGER_SOURCE_
    AUTO;
config.tiger_block[i].sources = bitmask;
```

struct timetagger4_channel*crono_bool_t* **enabled**

Enable the TDC channel.

crono_bool_t **rising**

Not applicable for the TimeTagger4.

Rising and/or falling edges are configured using *timetagger4_trigger*.

uint32_t **start**

Start value for the grouping functionality.

In multiples of *timetagger4_param_info.binsize*.

See also *stop* and *Grouping and Events*.

 Attention

For continuous mode, *timetagger4_channel.start* should be kept at its default value (as provided by *timetagger4_get_default_configuration()*).

uint32_t **stop**

Stop value for the grouping functionality.

In multiples of *timetagger4_param_info.binsize*.

Only hits between *start* and *stop* are read out.

For Gen 1, the range is $0 \leq \text{start} \leq \text{stop} \leq 2^{31}$.

For Gen 2, the range is $0 \leq \text{start} \leq \text{stop} \leq 2^{32}$.

See also *Grouping and Events*.

 Attention

For continuous mode, `timetagger4_channel.start` should be kept at its default value (as provided by `timetagger4_get_default_configuration()`).

struct `timetagger4_delay_config`

Only available for TimeTagger4 Gen 2.

uint32_t `delay`

Delay the corresponding input channel.

In units of `timetagger4_static_info.delay_bin_size`.

The range is $0 \leq \text{delay} \leq 1023$.

See also [Configurable Input Delay](#).

Runtime Control

Once a TimeTagger4 device is configured, the following functions can be used to control the behavior of it.

These functions return quickly with very little overhead. However, they are not guaranteed to be thread safe.

int **timetagger4_start_capture**(*timetagger4_device* *device)

Start data acquisition.

Note

`timetagger4_start_capture()` will reset the memory buffer.

If you wish to stop recording data and resume later without clearing the buffer, use `timetagger4_pause_capture()` and `timetagger4_continue_capture()`.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: `TIMETAGGER4_OK`, `TIMETAGGER4_INVALID_DEVICE`,
`TIMETAGGER4_CRONO_INTERNAL_ERROR`, `TIMETAGGER4_HARDWARE_FAILURE`,
or `TIMETAGGER4_WRONG_STATE`.

int **timetagger4_pause_capture**(*timetagger4_device* *device)

Pause data acquisition.

`timetagger4_pause_capture()` and `timetagger4_continue_capture()` have less overhead than `timetagger4_start_capture()` and `timetagger4_stop_capture()`, but do not allow for a configuration change.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: `TIMETAGGER4_OK`, `TIMETAGGER4_INVALID_DEVICE`,
`TIMETAGGER4_WRONG_STATE`.

int **timetagger4_continue_capture**(*timetagger4_device* *device)

Continue data acquisition.

`timetagger4_pause_capture()` and `timetagger4_continue_capture()` have less overhead than `timetagger4_start_capture()` and `timetagger4_stop_capture()`, but do not allow for a configuration change.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: `TIMETAGGER4_OK`, `TIMETAGGER4_INVALID_DEVICE`,
`TIMETAGGER4_WRONG_STATE`.


```
int timetagger4_stop_capture(timetagger4_device *device)
```

Stop data acquisition.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: *TIMETAGGER4_OK*, or *TIMETAGGER4_INVALID_DEVICE*.

```
int timetagger4_start_tiger(timetagger4_device *device)
```

Start the *Timing Generator (TiGer)*

This can be done independently of the state of the data acquisition.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: *TIMETAGGER4_OK*, *TIMETAGGER4_INVALID_DEVICE*, or *TIMETAGGER4_WRONG_STATE*.

```
int timetagger4_stop_tiger(timetagger4_device *device)
```

Stop the *Timing Generator (TiGer)*

This can be done independently of the state of the data acquisition.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

Status code: *TIMETAGGER4_OK* or *TIMETAGGER4_INVALID_DEVICE*.

Data Readout

The device provides a stream of *packets* (see [Output Data Format](#)) that are read in batches with `timetagger4_read()`.

`timetagger4_read()` requires `timetagger4_read_in` as configuration.

`timetagger4_read()` stores the read batch in a `timetagger4_read_out` struct.

Every packet needs to be *acknowledged* before its memory location can be used again by the TimeTagger4 device.

A typical workflow in an application would be:

- Start data acquisition (see [Runtime Control](#)).
- Read a batch of packets.
- Iterate through the batch and process the packets (see [Output Data Format](#) for the data layout). For this purpose, `crono_next_packet` is provided.
- Acknowledge the batch as processed with `timetagger4_acknowledge()`.

Instead of manually acknowledging each batch, each read batch of data can be automatically acknowledged by setting `timetagger4_read_in.acknowledge_last_read` to true.

The following example highlights the workflow. A complete coding example can be found on github.com/cronologic-de/xtdc_babel or in Section [Code Example](#).

```
timetagger_read_in read_config;
read_config.acknowledge_last_read = true;
timetagger_read_out read_data;

status = timetagger4_read(device, &read_config, &read_data)
if (status != TIMETAGGER4_OK) {
    // handle errors of the call to timetagger4_read
}
else if (read_data.error_code != CRONO_READ_OK)
{
    // handle read errors, e.g., empty packets
}
else
{
    volatile crono_packet *p = read_data.first_packet;
    while (p <= read_data.last_packet)
    {

        /* process data */

        p = crono_next_packet(p);
    }
}
```

```
int timetagger4_read(timetagger4_device *device, timetagger4_read_in *in, timetagger4_read_out
                    *out)
```

Read captured data.

Data is read in batches of *packets*. Pointers to the first and last packets are stored in *out.first_packet* and *out.last_packet*.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **in** – Pointer to a *timetagger4_read_in* struct, configuring the read call.
- **out** – Pointer to a *timetagger4_read_out* that will be filled by the call.

Returns

Status code: Same as *out.error_code*, or *TIMETAGGER4_INVALID_DEVICE*.

```
int timetagger4_acknowledge(timetagger4_device *device, volatile crono_packet *packet)
```

Acknowledge all packets up to packet.

Only acknowledged data can be overwritten by the DMA controller.

Explicitly calling *timetagger4_acknowledge* is only necessary if *timetagger4_read()* was called with *in.acknowledge_last_read == false*.

Note

timetagger4_acknowledge allows freeing up memory early if there will be no call to *timetagger4_read()* anytime soon.

Note

timetagger4_acknowledge allows keeping data over multiple calls of *timetagger4_read()*, avoiding unnecessary copying of data.

Attention

After acknowledging a packet, it becomes *immediately* invalid. It is *immediately* unsafe to attempt accessing its content.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **packet** – Pointer to a packet. All packets up to this one will be acknowledged.

Returns

Status code: *TIMETAGGER4_OK*, or *TIMETAGGER4_INVALID_DEVICE*.

```
crono_next_packet(current_packet)
```

Convenience macro that jumps to the next *crono_packet*.

⚠ Attention

You must explicitly check that the pointer to the next packet is valid!

Parameters

- **current_packet** – Pointer to the current packet.

Returns

Pointer to the next packet.

struct **timetagger4_read_in**

crono_bool_t **acknowledge_last_read**

Automatically acknowledge last readout.

If set, *timetagger4_read()* automatically acknowledges packets from the last read.

Otherwise, *timetagger4_acknowledge()* needs to be explicitly called by the user.

struct **timetagger4_read_out**

crono_packet ***first_packet**

Pointer to the first packet that was captured by the call of *timetagger4_read()*.

crono_packet ***last_packet**

Pointer to the last packet that was captured by the call of *timetagger4_read()*.

This packet is still valid. All data after this packet is invalid.

int **error_code**

Error codes.

One of the following:

CRONO_READ_OK

Evaluates to 0. Read was successful. No errors occurred.

CRONO_READ_NO_DATA

Evaluates to 1. The read attempt did not yield any data.

The pointers *timetagger4_read_out.first_packet* and *timetagger4_read_out.last_packet* will not point to valid packets in memory.

CRONO_READ_INTERNAL_ERROR

Evaluates to 2. Some unhandled error occurred. The TimeTagger4 device needs to be reinitialized.

CRONO_READ_TIMEOUT

Evaluates to 3. This error code is currently not used.

CRONO_READ_WRONG_STATE

Evaluates to 4. The TimeTagger4 is in a wrong state (see *Runtime Control*).

CRONO_READ_INVALID_ARGUMENTS

Evaluates to 5.

`const char *error_message`

Error message in human-readable form.

This may contain additional information about [error_code](#).

Output Data Format

struct `crono_packet`

Data read from `timetagger4_read()` is provided as *packets*. The data layout of a packet is *crono_packet*.

The following code snippets shows the principal of how to unpack a *crono_packet*.

A complete coding example can be found on github.com/cronologic-de/xtdc_babel or in Section [Code Example](#).

```
uint32_t binsize_ps = param_info->packet_binsize;

int64_t absolute_time_bins = packet->timestamp;
double absolute_time_ps = absolute_time_bins * binsize_ps;

// 2 TDC hits are in each packet->data word.
int hit_count = 2 * packet->length;
// If an odd number of hits were recorded, the high 32 bits of the last
// data word do not contain valid data
if ((packet->flags & TIMETAGGER4_PACKET_FLAG_ODD_HITS) != 0)
    hit_count -= 1;

uint32_t *packet_data = (uint32_t*)(packet->data);

// keep track of possible timestamp rollovers
uint32_t rollover_counter = 0;
uint64_t rollover_period_bins = static_info->rollover_period;

for(int i=0; i<hit_count; ++i)
{
    uint32_t channel = packet_data[i] & 0xF;
    uint32_t flags = packet_data[i] >> 4 & 0xF;

    if ((flags & TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW) != 0)
    {
        rollover_counter += 1;
    }
    else
    {
        uint32_t timestamp_bins = packet_data[i] >> 8 & 0xFFFFF;
        double timestamp_ps = binsize * (
            timestamp_bins + rollover_counter * rollover_period_bins);
        double absolute_timestamp = absolute_time_ps + timestamp_ps;
    }
}
```

uint8_t `channel`

Always 0 for the TimeTagger4.

The channel source of a hit are encoded in *data*.

uint8_t `card`

Identifies the source card of the data, in case multiple TimeTagger4 are installed in the system.

Defaults to 0 if no value is assigned to `timetagger4_init_parameters.board_id`.

uint8_t type

The type of `crono_packet.data`.

In case of the TimeTagger4, this is always an 32-bit unsigned integer, corresponding to `CRONO_PACKET_TYPE_32_BIT_UNSIGNED`.

uint8_t flags

Bit field of packet flags.

Each bit corresponds to one of the following:

TIMETAGGER4_PACKET_FLAG_ODD_HITS

0x1.

If this bit is set, the last word in the `data` array consists of one timestamp only, which means that only the lower 32 bits (little endian) of the 64-bit data word are valid.

TIMETAGGER4_PACKET_FLAG_SLOW_SYNC

0x2.

If this bit is set, the timestamp of a hit is above the range of the 8-bit rollover number and the 24-bit hit timestamp. The group is closed and all other hits are ignored.

TIMETAGGER4_PACKET_FLAG_START_MISSED

0x4.

If this bit is set, packets were discarded due to a full FIFO potentially caused by a data hit rate that is too high.

Starts were missed and stops are potentially in wrong groups.

TIMETAGGER4_PACKET_FLAG_SHORTENED

0x8.

If this bit is set, packets were shortened due to a full pipeline of the FIFO potentially caused by a data hit rate that is too high.

Stops are missing in the current packet.

TIMETAGGER4_PACKET_FLAG_DMA_FIFO_FULL

0x10.

If this bit is set, the internal FIFO was full, potentially caused by a hit rate that is too high.

TIMETAGGER4_PACKET_FLAG_HOST_BUFFER_FULL

0x20.

If this bit is set, the host buffer was full. This might result in dropped packets.

This is caused either by a hit rate that is too high or by data not being retrieved and acknowledged fast enough from the buffer.

Solutions are increasing the buffer size (see `timetagger4_init_parameters.buffer_size`) or avoiding or optimizing any additional processing in the application reading the data.

uint32_t length

Length of *data* in multiples of 64 bit.

Each 64-bit data word contains up to two TDC hits.

The number of TDC hits can be calculated as

$$2 \times \text{length} - ((\text{flags} \& \text{TIMETAGGER4_PACKET_FLAG_ODD_HITS}) ? 1 : 0)$$

uint64_t timestamp

Coarse timestamp of the Start trigger hit. Values are given in multiples of *timetagger4_param_info.packet_binsize*.

uint64_t data[1]

Contains the TDC hits as variable length array.

The length is given by *length* and can be zero.

Note

To directly operate on the TDC hit, cast this array to `uint32_t`.

The data layout of a 32-bit TDC hit is as follows:

Bits	31 – 8	7 – 4	3 – 0
Content	24-bit timestamp	Flags	Channel

That is, one can extract the timestamp, flags, and channel number as follows:

```
uint32_t timestamp = (hit >> 8) & 0xFFFFF;
uint32_t flags      = (hit >> 4) & 0xF;
uint32_t channel    = hit      & 0xF;
```

The 24-bit timestamp is given in multiples of *timetagger4_param_info.packet_binsize*.

Channel value 0–3 correspond to Stop channels A–D.

The flag bits correspond to the following:

TIMETAGGER4_HIT_FLAG_COARSE_TIMESTAMP

0x4.

This bit is always set for the TimeTagger4.

TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW

0x2.

If this bit is set, the data word indicates a rollover and does *not* encode a TDC hit.

The time since the Start trigger hit exceeded the 24-bit range that can be encoded in one 32-bit TDC hit.

To correctly determine the timestamp of the following TDC hits, the application needs to increment a rollover counter if the TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW bit is set:

```
int rollover_counter = 0;
if ((flags & TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW) != 0)
{
    rollover_counter += 1;
}
timestamp += rollover_counter * rollover_period
```

rollover_period is given by *timetagger4_static_info.rollover_period*.

 Attention

The rollover counter needs to be reset for each *crono_packet*.

TIMETAGGER4_HIT_FLAG_RISING

0x1.

If this bit is set, this hit corresponds to a signal's rising edge, otherwise to a falling edge.

Status Information

```
int timetagger4_get_device_type(timetagger4_device *device)
```

Get the type of the TimeTagger device.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

CRONO_DEVICE_TIMETAGGER4

```
const char *timetagger4_get_last_error_message(timetagger4_device *device)
```

Get the last error message.

Parameters

- **device** – Pointer to a TimeTagger4 device.

Returns

The error message.

```
int timetagger4_get_fast_info(timetagger4_device *device, timetagger4_fast_info *info)
```

Obtain dynamic information about a TimeTagger4 device that can be obtained within a few microseconds.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **info** – Pointer to a *timetagger4_fast_info* struct that will be filled.

Returns

Status codes: *TIMETAGGER4_OK*, *TIMETAGGER4_INVALID_DEVICE*, *TIMETAGGER4_CRONO_INVALID_ARGUMENTS*, or -1.

```
int timetagger4_get_param_info(timetagger4_device *device, timetagger4_fast_info *info)
```

Obtain information about configuration changes.

Gets information that changes indirectly due to configuration changes.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **info** – Pointer to a *timetagger4_param_info* struct that will be filled.

Returns

Status codes: *TIMETAGGER4_OK*, *TIMETAGGER4_CRONO_INVALID_ARGUMENTS*, or *TIMETAGGER4_WRONG_STATE*,

```
int timetagger4_get_static_info(timetagger4_device *device, timetagger4_static_info *info)
```

Obtain information about a TimeTagger4 device that does not change during runtime.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **info** – Pointer to a *timetagger4_static_info* struct that will be filled.

Returns

Status codes: *TIMETAGGER4_OK*, *TIMETAGGER4_INVALID_DEVICE*, or *TIMETAGGER4_CRONO_INVALID_ARGUMENTS*.

```
int timetagger4_get_pcie_info(timetagger4_device *device, crono_pcie_info *pcie_info)
```

Obtain PCIe information.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **pcie_info** – Pointer to a *crono_pcie_info* struct that will be filled.

Returns

Status codes: *TIMETAGGER4_OK*, *TIMETAGGER4_INVALID_DEVICE*, or *TIMETAGGER4_HARDWARE_FAILURE*.

```
int timetagger4_clear_pcie_errors(timetagger4_device *device, int flags)
```

Clear PCIe errors.

Only useful for PCIe debugging.

Valid flags are:

CRONO_PCIE_CORRECTABLE_FLAG

Clear correctable PCIe errors.

CRONO_PCIE_UNCORRECTABLE_FLAG

Clear uncorrectable PCIe errors.

Parameters

- **device** – Pointer to a TimeTagger4 device.
- **flag** – Flag which errors to clear.

Returns

Status codes: *TIMETAGGER4_OK*, or *TIMETAGGER4_INVALID_DEVICE*.

```
struct timetagger4_fast_info
```

int **size**

The number of bytes this struct occupies.

int **version**

Version number of this struct. It is increased when the definition of the struct changes.

int **tdc_rpm**

Speed of the TDC fan in rounds per minute.

0 if no fan is present.

int **fpga_rpm**

Speed of the FPGA fan in rounds per minute.

0 if no fan is present.

int **alerts**

Alert bits from the temperature sensor and the system monitor.

The TimeTagger4 does not implement any temperature alerts.

int **pcie_pwr_mgmt**

Fixed at 0.

int **pcie_link_width**

Number of PCIe lanes the TimeTagger4 device uses.

Should always be 10.

int **pcie_max_payload**

Maximum size in bytes for one PCIe transaction.

Depends on the system configuration.

struct **timetagger4_param_info**

int **size**

The number of bytes this struct occupies.

int **version**

Version number of this struct. It is increased when the definition of the struct changes.

double **binsize**

Bin size in ps of the measured TDC data.

For Gen 1 boards, this is 500 ps.

For Gen 2 boards, this is 100 ps.

int **board_id**

ID of the TimeTagger4 device.

Used to identify itself in the output data stream.

Is between 0 and 255.

int **channels**

Number of TDC channels of the board.

Fixed at 4.

int **channel_mask**

Bit assignment of each enabled input channel.

Bit $0 \leq n \leq 3$ is set if channel n is enabled.

int64_t **total_buffer**

The total amount of DMA buffer in bytes.

double **packet_binsize**

The binsize used for the TimeTagger4.

The TDC data timestamps of [crono_packet.data](#) is given in multiples of this value.

For Gen 1 boards, this is 500 ps.

For Gen 2 boards, this is 100 ps.

double quantisation

Quantisation, that is, measurement resolution of the TimeTagger4, in ps.

Depends on the board variant:

-1G	-2G	-1.25G	-2.5G	-5G	-10G
1000	500	800	400	200	100

That means that, e.g., for the -1.25G, the lower 3 bits of the timestamps of `crono_packet.data` are 0.

struct timetagger4_static_info

Structure that contains static information.

This structure contains information about the board that does not change during run time. It is provided by the function `timetagger4_get_static_info()`.

int size

The number of bytes occupied by the structure.

int version

The version number.

A version number that is increased when the definition of the structure is changed. The increment can be larger than one to match driver version numbers or similar. Set to 0 for all versions up to first release.

int board_id

ID of the TimeTagger4 board.

This value is passed to the constructor. It is reflected in the output data.

int driver_revision

Encoded version number.

The lower three bytes contain a triple level hierarchy of version numbers. E.g. 0x010103 codes version 1.1.3.

The versioning follows the [Semantic Versioning 2.0.0](#) specifications.

int driver_build_revision

SVN revision of the driver build.

int firmware_revision

Revision number of the FPGA configuration.

int board_revision

Board revision number.

The board revision number can be read from a register. It is a four bit number that changes when the schematic of the board is changed.

- 0: Experimental first board Version. Labeled "Rev. 1"
- 2: First commercial Version. Labeled "Rev. 2"

int board_configuration

Describes the schematic configuration of the board.

The same board schematic can be populated in multiple variants. This is a eight bit code that can be read from a register.

int subversion_revision

Subversion revision ID of the FPGA configuration.

int chip_id

Reserved.

int board_serial

Serial number of the TimeTagger4.

With year and running number in 8.24 format (8 bits are used to encode the year, 24 bits are used to encode the serial number). The number is identical to the one printed on the silvery sticker on the board.

unsigned int flash_serial_low

Low 32 bits of the 64-bit manufacturer serial number of the flash chip.

unsigned int flash_serial_high

High 32 bits of the 64-bit manufacturer serial number of the flash chip.

***crono_bool_t* flash_valid**

Flash data is valid and in use.

If `true`, the driver found valid calibration data in the flash on the board and is using it.

char calibration_date

Calibration date.

DIN EN ISO 8601 string YYYY-MM-DD HH:DD describing the time when the card was calibrated.

char bitstream_date

Bitstream creation date.

DIN EN ISO 8601 string YYYY-MM-DD HH:DD:SS describing the time when the bitstream was created.

double delay_bin_size

Bin size of *timetagger4_delay_config.delay* in ps.

double auto_trigger_ref_clock

Auto trigger clock frequency in Hz.

Used for calculating the frequency of the auto trigger (see *Auto-Triggering Function Generator* and *timetagger4_configuration.auto_trigger_period*).

uint32_t rollover_period

The number of bins in a rollover period.

If a rollover occurred, *T* bins have to be added to the timestamp of the hit (see *crono_packet.data* for more information, in particular, *TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW*)

struct crono_pcie_info**uint32_t pwr_mgmt**

Organizes power supply of the PCIe lanes.

uint32_t link_width

Number of PCIe lanes that the card uses.

uint32_t max_payload

Maximum size in bytes for one PCIe transaction.

uint32_t link_speed

Data rate of the PCIe card.

uint32_t error_status_supported

Different from 0 if the PCIe error status is supported for this device.

uint32_t correctable_error_status

Correctable error status flags.

Read directly from the PCIe config register.

Useful for debugging PCIe problems.

One of the following:

CRONO_PCIE_RX_ERROR

Equals 1 << 0.

CRONO_PCIE_BAD_TLP

Equals 1 << 6.

CRONO_PCIE_BAD_DLLP

Equals 1 << 7.

CRONO_PCIE_REPLAY_NUM_ROLLOVER

Equals 1 << 8.

CRONO_PCIE_REPLAY_TIMER_TIMEOUT

Equals 1 << 12.

CRONO_PCIE_ADVISORY_NON_FATAL

Equals 1 << 13.

CRONO_PCIE_CORRECTED_INTERNAL_ERROR

Equals 1 << 14.

CRONO_PCIE_HEADER_LOG_OVERFLOW

Equals 1 << 15.

uint32_t uncorrectable_error_status

Uncorrectable error status flags.

Read directly from the PCIe config register.

Useful for debugging PCIe problems.

One of the following:

CRONO_PCIE_UNC_UNDEFINED

Equals 1 << 0.

CRONO_PCIE_UNC_DATA_LINK_PROTOCOL_ERROR

Equals 1 << 4.

CRONO_PCIE_UNC_SURPRISE_DOWN_ERROR

Equals 1 << 5.

CRONO_PCIE_UNC_POISONED_TLP

Equals 1 << 12.

CRONO_PCIE_UNC_FLOW_CONTROL_PROTOCOL_ERROR

Equals 1 << 13.

CRONO_PCIE_UNC_COMPLETION_TIMEOUT

Equals 1 << 14.

CRONO_PCIE_UNC_COMPLETER_ABORT

Equals 1 << 15.

CRONO_PCIE_UNC_UNEXPECTED_COMPLETION

Equals 1 << 16.

CRONO_PCIE_UNC_RECEIVER_OVERFLOW_ERROR

Equals 1 << 17.

CRONO_PCIE_UNC_MALFORMED_TLP

Equals 1 << 18.

CRONO_PCIE_UNC_ECRC_ERROR

Equals 1 << 19.

CRONO_PCIE_UNC_UNSUPPORTED_REQUEST_ERROR

Equals 1 << 20.

Driver Information

int **timetagger4_get_driver_revision()**

Get the driver revision of the installed TimeTagger4 driver.

This function does not require a TimeTagger4 board to be installed in the system.

Returns

The driver version in the same format as

timetagger4_static_info.driver_revision.

const char ***timetagger4_get_driver_revision_str()**

Get the driver revision of the installed TimeTagger4 driver.

This function does not require a TimeTagger4 board to be installed in the system.

Returns

The driver version including the SVN build revision as a string.

int **timetagger4_count_devices**(int *error_code, char **error_message)

Count the number of boards installed in the system that are supported by the installed TimeTagger4 driver.

Parameters

- **error_code** – Pointer to the location where a potential error code will be stored. Equals *TIMETAGGER4_OK* if no error occurred.
- **error_message** – Pointer to a location where a potential error message in plain text will be stored.

Returns

The number of boards.

4 Code Example

The following C++ source code shows how to initialize a TimeTagger4 board, configure it, and loop over incoming packets.

The source code can also be found on our GitHub repository, located at https://github.com/cronologic-de/xtdc_babel/tree/main/timetagger4_user_guide_example.

```

1 // timetagger4_user_guide_example.cpp : Example application for the TimeTagger4
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <chrono>
5 #include <thread>
6 #include "TimeTagger4_interface.h"
7
8 // If true the time tagger triggers a start periodically
9 // The time difference of signals on channel A are measured
10 // else start signal either from input or tiger is used (see below)
11 // frequency of start signal is printed and the hits are sampled
12 const bool USE_CONTINUOUS_MODE = false;
13 const bool USE_TIGER_START = true;           // if false, external signal must be
14 // provided on start; not applicable if continuous mode is enabled
15 const bool USE_TIGER_STOPS = true;           // if false please connect signals to
16 // some of channels A-D
17
18 timetagger4_device * initialize_timetagger(int buffer_size, int board_id, int
19 // card_index) {
20 // prepare initialization
21 timetagger4_init_parameters params;
22
23 timetagger4_get_default_init_parameters(&params);
24 params.buffer_size[0] = buffer_size;           // size
25 // of the packet buffer
26 params.board_id = board_id;
27 // value copied to "card" field of every packet, allowed range 0..255
28 params.card_index = card_index;
29 // which of the TimeTagger4 board found in the system to be used
30
31 int error_code;
32 const char * err_message;
33 timetagger4_device * device = timetagger4_init(&params, &error_code, &err_
34 // message);
35 if (error_code != CRONO_OK) {
36     printf("Could not init TimeTagger4 compatible board: %s\n", err_message);
37     return nullptr;
38 }
39 timetagger4_static_info static_info;
40 timetagger4_get_static_info(device, &static_info);

```

(continues on next page)

(continued from previous page)

```

35     bool timeTaggerNG = static_info.board_revision >= 7;
36     if (USE_CONTINUOUS_MODE && !timeTaggerNG) {
37         printf("Cannot use continuous mode with TimeTagger 1G/2G: %s\n", err_
←message);
38         timetagger4_close(device);
39         return nullptr;
40     }
41
42     return device;
43 }
44
45 int configure_timetagger(timetagger4_device * device) {
46     // prepare configuration
47     timetagger4_static_info static_info;
48     timetagger4_get_static_info(device, &static_info);
49     timetagger4_configuration config;
50     // fill configuration data structure with default values
51     // so that the configuration is valid and only parameters
52     // of interest have to be set explicitly
53     timetagger4_get_default_configuration(device, &config);
54
55     // set config of the 4 TDC channels
56     for (int i = 0; i < TIMETAGGER4_TDC_CHANNEL_COUNT; i++)
57     {
58         // enable recording hits on TDC channel
59         config.channel[i].enabled = true;
60
61         // define range of the group
62         config.channel[i].start = 0;           // range begins right after start_
←pulse
63         if (!USE_CONTINUOUS_MODE) {
64             config.channel[i].stop = 30000;    // recording window stops_
←after ~15 us
65         }
66         else {
67             config.channel[i].stop = 0x7fffffff; // trigger is independent of_
←stops
68             // set to maximal value
69         }
70
71         // measure only rising edge for tiger (positive) pulse or falling for_
←user (negative) pulse
72         config.trigger[TIMETAGGER4_TRIGGER_A + i].falling = USE_TIGER_STOPS ? 0_
←: 1;
73         config.trigger[TIMETAGGER4_TRIGGER_A + i].rising = USE_TIGER_STOPS ? 1 :_
←0;
74     }
75
76     // generate an internal 25 kHz trigger, used for tiger and continuous mode
77     config.auto_trigger_period = (int)(static_info.auto_trigger_ref_clock /_

```

(continues on next page)

```

25000);
78     config.auto_trigger_random_exponent = 0;
79
80     // setup TiGeR
81     // sending a signal to the LEMO outputs (and to the TDC on the same channel)
82     // requires proper 50 Ohm termination on the LEMO output to work reliably
83
84     // width of the 12ns pulse in the auto_trigger clock periods
85     int pulse_width = (int) (12e-9 * static_info.auto_trigger_ref_clock);
86
87     if (!USE_CONTINUOUS_MODE) {
88         // use 200 kHz auto trigger to generate
89
90         // generate above configured auto trigger to generate a
91         // signal with 12 ns pulse width on LEMO output Start
92         config.tiger_block[0].enable = USE_TIGER_START ? 1 : 0;
93         config.tiger_block[0].start = 0;
94         config.tiger_block[0].stop = config.tiger_block[0].start + pulse_width;
95         config.tiger_block[0].negate = 0;
96         config.tiger_block[0].retrigger = 0;
97         config.tiger_block[0].extend = 0;
98         config.tiger_block[0].enable_lemo_output = 1;
99         config.tiger_block[0].sources = TIMETAGGER4_TRIGGER_SOURCE_AUTO;
100        // if TiGeR is used for triggering with positive pulses
101        if (USE_TIGER_START)
102            config.dc_offset[0] = TIMETAGGER4_DC_OFFSET_P_LVCMOS_18;
103        else // user input expect NIM signal
104            config.dc_offset[0] = TIMETAGGER4_DC_OFFSET_N_NIM;
105
106        // start group on falling edges on the start channel 0
107        config.trigger[TIMETAGGER4_TRIGGER_S].falling = USE_TIGER_START ? 0 : 1;
108        config.trigger[TIMETAGGER4_TRIGGER_S].rising = USE_TIGER_START ? 1 : 0;
109    } else {
110        // Auto trigger is used as a start signal
111        config.tdc_mode = TIMETAGGER4_TDC_MODE_CONTINUOUS;
112    }
113
114    for (int i = 1; i < TDC4_TIGER_COUNT; i++) {
115        config.tiger_block[i].enable = USE_TIGER_STOPS ? 1 : 0;
116        config.tiger_block[i].start = i * 100;
117        config.tiger_block[i].stop = config.tiger_block[i].start + pulse_width;
118        config.tiger_block[i].negate = 0;
119        config.tiger_block[i].retrigger = 0;
120        config.tiger_block[i].extend = 0;
121        config.tiger_block[i].enable_lemo_output = USE_TIGER_STOPS ? 1 : 0;
122        config.tiger_block[i].sources = TIMETAGGER4_TRIGGER_SOURCE_AUTO;
123
124
125        if (USE_TIGER_STOPS)
126            config.dc_offset[i] = TIMETAGGER4_DC_OFFSET_P_LVCMOS_18;

```

(continues on next page)

```

127     else // user input expect NIM signal
128         config.dc_offset[i] = TIMETAGGER4_DC_OFFSET_N_NIM;
129
130         // this is not related to the tigers, but uses the same indexing (0 is
131         // start)
132         // optionally increase input delay by 10 * 200 ps for each channel on
133         // new TT
134         // config.delay_config[i].delay = i * 10;
135     }
136
137     // write configuration to board
138     return timetagger4_configure(device, &config);
139 }
140
141 void print_device_information(timetagger4_device * device, timetagger4_static_
142     info * si, timetagger4_param_info * pi) {
143     // print board information
144     printf("Board Serial      : %d.%d\n", si->board_serial >> 24, si->board_
145     serial & 0xffffffff);
146     printf("Board Configuration : %s\n", timetagger4_get_device_name(device));
147     printf("Board Revision      : %d\n", si->board_revision);
148     printf("Firmware Revision   : %d.%d\n", si->firmware_revision, si->
149     subversion_revision);
150     printf("Driver Revision      : %d.%d.%d\n", ((si->driver_revision >> 16) &
151     255), ((si->driver_revision >> 8) & 255), (si->driver_revision & 255));
152     printf("Driver SVN Revision : %d\n", si->driver_build_revision);
153     printf("\nTDC binsize      : %0.2f ps\n", pi->binsize);
154 }
155
156 double last_abs_ts_on_a = 0;
157 int64_t last_group_abs_time = 0;
158
159 int64_t processPacket(volatile crono_packet *p, bool print, timetagger4_static_
160     info *si, timetagger4_param_info *pi) {
161     // do something with the data, e.g. calculate current rate
162     int64_t group_abs_time = p->timestamp;
163     if (!USE_CONTINUOUS_MODE) {
164         // group timestamp increments at binsize, but we see only a fraction of
165         // the packets (every update_count)
166         double rate = 1e12 / ((double)(group_abs_time - last_group_abs_time) *
167         pi->packet_binsize);
168         if (print && last_group_abs_time > 0) {
169             printf("\nr%.6f kHz", rate / 1000.0);
170             // ...or print hits (not a good idea at high data rates,
171             printf("Card %d - flags %d - length %d - type %d - TS %llu\n", p->
172             card, p->flags, p->length, p->type, p->timestamp);
173         }
174         last_group_abs_time = group_abs_time;
175     }
176 }

```

```

167
168     int hit_count = 2 * (p->length);
169     // Two hits fit into every 64 bit word. The second in the last word might be
empty
170     // This flag tells us, whether the number of hits in the packet is odd
171     if ((p->flags & TIMETAGGER4_PACKET_FLAG_ODD_HITS) != 0)
172         hit_count -= 1;
173
174     uint32_t* packet_data = (uint32_t*)(p->data);
175     uint32_t rollover_count = 0;
176     //
177     uint64_t rollover_period_bins = si->rollover_period;
178     for (int i = 0; i < hit_count; i++)
179     {
180         uint32_t hit = packet_data[i];
181         uint32_t channel = hit & 0xf;
182         // extract hit flags
183         uint32_t flags = hit >> 4 & 0xf;
184
185
186         if ((flags & TIMETAGGER4_HIT_FLAG_TIME_OVERFLOW) != 0) {
187             // this is a overflow of the 23/24 bit counter)
188             rollover_count++;
189         }
190         else {
191             // extract channel number (A-D)
192             char channel_letter = 65 + channel;
193
194             // extract hit timestamp
195             uint32_t ts_offset = hit >> 8 & 0xffffffff;
196
197             // Convert timestamp to ns, this is relative to the start of the
group
198             double ts_offset_ns = (ts_offset + rollover_count * rollover_period_
bins) * pi->binsize / 1000.0;
199
200             if (USE_CONTINUOUS_MODE) {
201                 if (channel == 0)
202                 {
203                     // compute the absolute time by adding the group time in ns
204                     double abs_ts_on_a = (group_abs_time * pi->packet_binsize) /
1000 + ts_offset_ns;
205                     double diff = abs_ts_on_a - last_abs_ts_on_a;
206                     if (last_abs_ts_on_a > 0 && print) {
207                         printf("Time difference between hits on A %.1f ns\n",
diff);
208                     }
209
210                     last_abs_ts_on_a = abs_ts_on_a;
211                 }

```

```

212     }
213     else {
214         if (print)
215             printf("Hit on channel %c - flags %d - offset %u (raw) / %.
216     if ns\n", channel_letter, flags, ts_offset, ts_offset_ns);
217     }
218 }
219 return group_abs_time;
220 }
221
222 int main(int argc, char* argv[]) {
223     printf("cronologic timetagger4_user_guide_example using driver: %s\n",
224     timetagger4_get_driver_revision_str());
225     timetagger4_device* device = initialize_timetagger(8 * 1024 * 1024, 0, 0);
226     if (device == nullptr) {
227         exit(1);
228     }
229     int status = configure_timetagger(device);
230     if (status != CRONO_OK) {
231         printf("Could not configure TimeTagger4: %s", timetagger4_get_last_error_
232     message(device));
233         timetagger4_close(device);
234         return status;
235     }
236     timetagger4_static_info static_info;
237     timetagger4_get_static_info(device, &static_info);
238
239     timetagger4_param_info parinfo;
240     timetagger4_get_param_info(device, &parinfo);
241
242     print_device_information(device, &static_info, &parinfo);
243
244     // configure readout behaviour
245     timetagger4_read_in read_config;
246     // automatically acknowledge all data as processed
247     // on the next call to timetagger4_read()
248     // old packet pointers are invalid after calling timetagger4_read()
249     read_config.acknowledge_last_read = 1;
250
251     // structure with packet pointers for read data
252     timetagger4_read_out read_data;
253
254     // start data capture
255     status = timetagger4_start_capture(device);
256     if (status != CRONO_OK) {
257         printf("Could not start capturing %s", timetagger4_get_last_error_
258     message(device));
259         timetagger4_close(device);
260         return status;

```

```

258     }
259
260     // start timing generator
261     timetagger4_start_tiger(device);
262
263     // some book keeping
264     int packet_count = 0;
265     int empty_packets = 0;
266     int packets_with_errors = 0;
267     bool last_read_no_data = false;
268
269     int64_t group_abs_time = 0;
270     int64_t group_abs_time_old = 0;
271     int update_count = 100;
272
273     printf("Reading packets:\n");
274     bool no_data_printed = false;
275     // read 10000 packets
276     while (packet_count < 10000)
277     {
278         // get pointers to acquired packets
279         status = timetagger4_read(device, &read_config, &read_data);
280         if (status != CRONO_OK) {
281             std::this_thread::sleep_for(std::chrono::milliseconds(10));
282             // to avoid a lot of lines with no data
283             if (!no_data_printed) {
284                 printf(" - No data! -\n");
285                 no_data_printed = true;
286             }
287         }
288         else
289         {
290             // iterate over all packets received with the last read
291             volatile crono_packet* p = read_data.first_packet;
292             while (p <= read_data.last_packet)
293             {
294                 // printf is slow, so this demo only processes every nth packet
295                 // your application would of course collect every packet
296                 bool print = packet_count % update_count == 0;
297
298                 processPacket( p, print, &static_info, &parinfo);
299                 no_data_printed = false;
300                 p = crono_next_packet(p);
301                 packet_count++;
302             }
303         }
304     }
305
306     // shut down packet generation and DMA transfers
307     timetagger4_stop_capture(device);

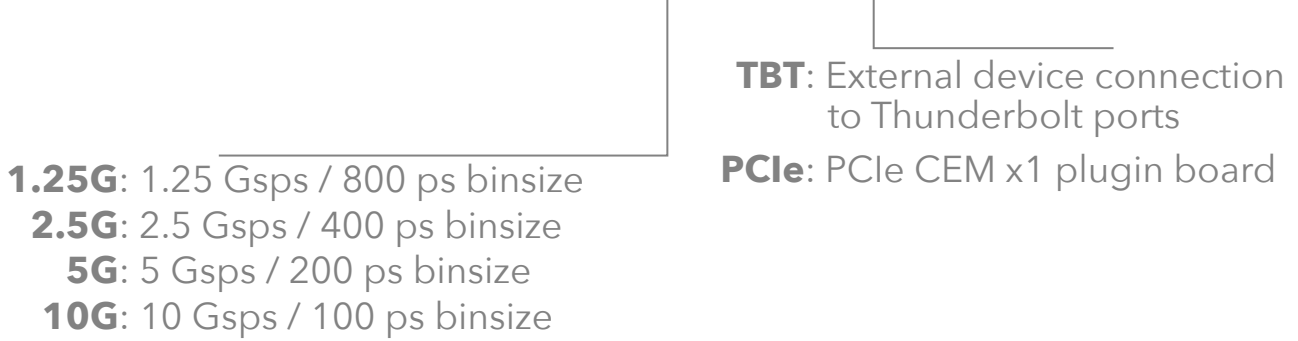
```


(continued from previous page)

```
308  
309 // deactivate timetagger4  
310 timetagger4_close(device);  
311 return 0;  
312 }
```

5 Ordering Information

TimeTagger4 – 1.25G – TBT



1.25G: 1.25 Gsps / 800 ps binsize

2.5G: 2.5 Gsps / 400 ps binsize

5G: 5 Gsps / 200 ps binsize

10G: 10 Gsps / 100 ps binsize

TBT: External device connection to Thunderbolt ports

PCle: PCIe CEM x1 plugin board

6 Technical Data

Each board is tested against the values listed in the columns “Min” and “Max”. “Typical” (Typ.) is the mean value of the first 10 boards produced or a value that is set by design.

Dimension

The TimeTagger4 boards fit any standard full-height PCIe slot and occupy a single slot width.

The dimensions, excluding the slot bracket, are **167.7 × 111.15 × 16.8 mm** (depths × height × width).

TDC Characteristics

TDC Measurement Characteristics for Gen 1

Symbol	Parameter	Min	Typ.	Max	Units
INL	Integral non-linearity			0.5	bins
DNL	Differential non-linearity		0.2		bins
t_{Data}	Data format bin size		500		ps
t_{Res1}	Double pulse resolution for -1G		1000		ps
t_{Res2}	Double pulse resolution for -2G		500		ps
Δt_{Start}	Interval between consecutive starts	4			ns
t_{Range}	Measurement range using hits only		8.368		ms
t_{Extended}	Extended range using rollovers		2.147		s
f_{Readout}	Readout rate			48	MHits/s

TDC Measurement Characteristics for Gen 2

Symbol	Parameter	Min	Typ.	Max	Units
INL	Integral non-linearity			0.5	bins
DNL	Differential non-linearity				t_{Data}
	-1.25G to -5G			0.01	
	-10G			0.3	
t_{Data}	Data format bin size		100		ps
t_{Quant}	Quantization				ps
	-1.25G		800		
	-2.5G		400		
	-5G		200		
	-10G		100		
t_{Res}	Double pulse resolution		2		t_{Quant}
t_{Range}	Measurement range using hits only			1.677	ms
t_{Extended}	Extended range using rollovers			0.429	s
$f_{\text{Start,burst}}$	Burst rate for up to 4096 starts				GHz
	-1.25G			0.625	
	-2.5G			1.25	
	-5G			2.5	
	-10G			5	
$f_{\text{Start,sust}}$	Sustained rate of starts			18	MHz
$f_{\text{Stop,burst}}$	Burst rate for up to 3900 stops				GHits/s
	-1.25G			0.625	
	-2.5G			1.25	
	-5G			2.5	
	-10G			5	
$f_{\text{Readout,single}}$	Readout rate per channel			40	MHits/s
$f_{\text{Readout,all}}$	Readout rate of all channels			60	MHits/s

Oscillator Time Base

Symbol	Parameter	Min	Typ.	Max	Units
ΔT	Temperature stability 20°C to 70°C			250	ppb
F	Initial calibration			1	ppm
$\Delta F/F_1$	Aging first year			2	ppm
$\Delta F/F_{10}$	Aging 10 years			8	ppm

Cable Delay Test

Figure 6.1 shows example histograms resulting from a cable delay test, in which a constant delay is measured multiple times. From these histograms it is apparent that the practically the only contribution to the timing error the quantization error (see our [application note](#) on the topic).

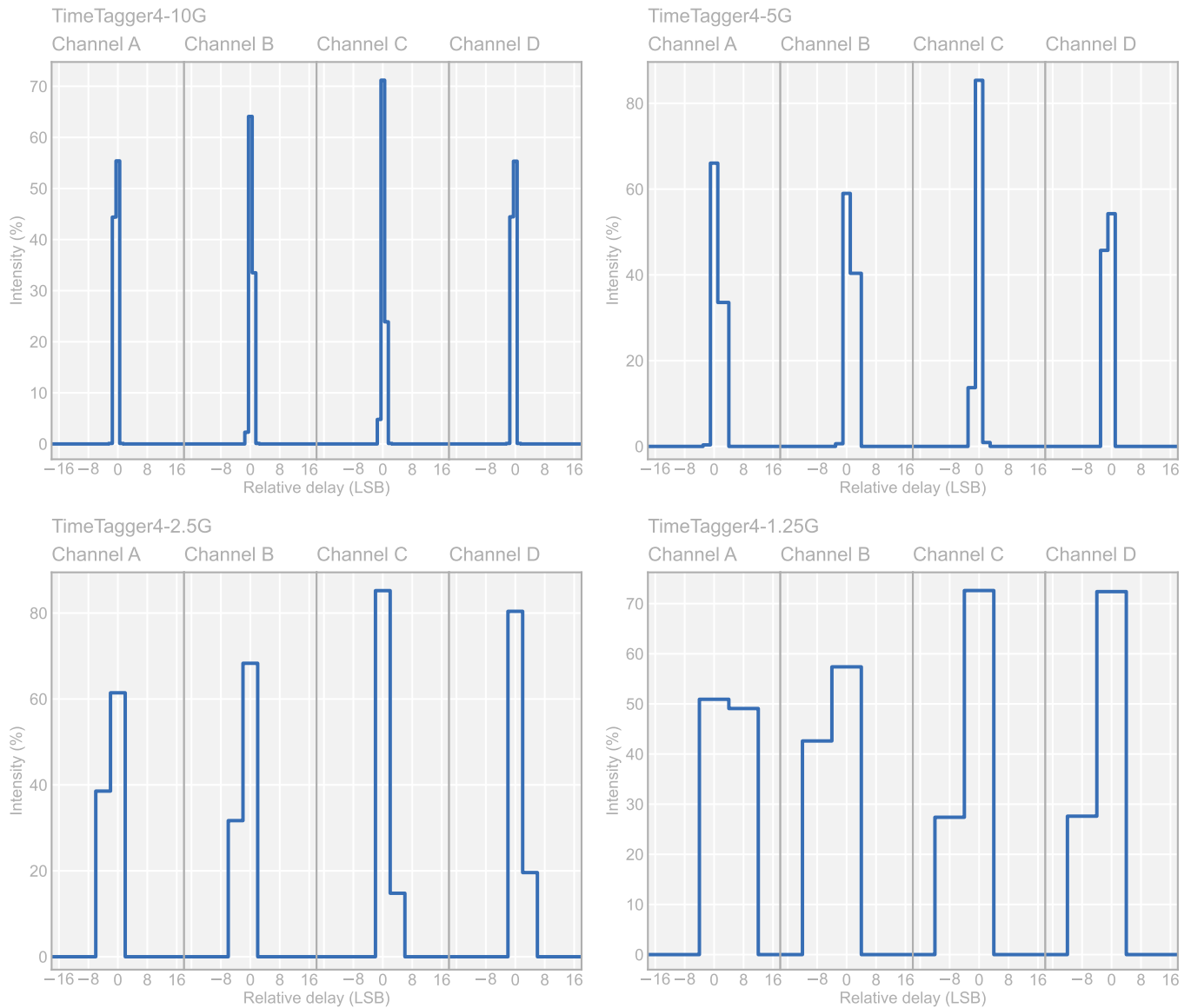


Figure 6.1 Histograms of a cable delay test for all TimeTagger4 variants. Each histogram is shifted such that the highest bin is located at zero. The x-axis is in units of LSB, with 1 LSB = 100 ps (see [packet_binsize](#)).

Electrical Characteristics

Power Supply

Symbol	Parameter	Min	Typ.	Max	Units
P_{total}	Total power consumption			10	W
$VCC_{3.3}$	PCIe 3.3 V rail power supply voltage	3.1	3.3	3.5	V
$I_{3.3}$	PCIe 3.3 V rail input current			650	mA
VCC_{12}	PCIe 12 V rail power supply voltage	11.1	12.0	12.9	V
I_{12}	PCIe 12 V rail input current			550	mA
VCC_{aux}	PCIe 3.3 V _{Aux} rail power supply voltage		3.3		V
I_{aux}	PCIe 3.3 V _{Aux} rail input current		0		mA

TDC Inputs

The TimeTagger4's inputs are single-ended AC-coupled with 50 Ω termination.

All inputs are AC-coupled. The inputs have very high input bandwidth requirements and therefore there are no circuits that provide over-voltage protection for these signals.

Keep in mind, that the input baseline V_{Base} is affected by the ratio of pulse length t_{Pulse} to average pulse distance (for continuous signals the term is called duty cycle).

Attention

Any voltage on the inputs above **5 V** or below **-5 V** relative to the voltage of the slot cover can result in permanent damage to the board.

⚠ Attention

Make sure not to drive the inputs when the connector is configured as a *TiGer* output.

Symbol	Parameter	Min	Typ.	Max	Units
V_{Base}	Input Baseline ¹	-5		5	V
$V_{\text{Threshold}}$	Trigger Level	$V_{\text{Base}} - 1.27$		$V_{\text{Base}} + 1.13$	V
$ \Delta V_{\text{min}} $	Magnitude of minimal pulse height	10			mV
t_{Pulse}	Pulse Length				ns
	-1.25G	0.8			
	-2.5G	0.4	5	200	
	-5G	0.2			
	-10G	0.1			
t_{Rise}	Pulse Edge 20% to 80%			10	ns
t_{Fall}	Pulse Edge 80% to 20%			10	ns
Z_p	Input Impedance		50		Ω
I_{Term}	Termination Current	-50	-20	50	mA

¹ Be wary that the total input voltage (baseline + pulse) does not exceed ± 5 V!

Information Required by DIN EN 61010-1

Manufacturer

The TimeTagger4 is a product of:

cronologic GmbH & Co. KG
 Jahnstraße 49
 D-60318 Frankfurt

Germany HRA 42869 beim Amtsgericht Frankfurt/M
 VAT-ID: DE235184378
 PCI Vendor ID: 0x1A13

Intended Use and System Integration

The devices are not ready to use as delivered by cronologic. It requires the development of specialized software to fulfill the application of the end-user. The device is provided to system integrators to be built into measurement systems that are distributed to end users. These systems usually consist of the TimeTagger4, a main board, a case, application software and possibly additional electronics to attach the system to some type of detector. They might also be integrated with the detector.

The TimeTagger4 is designed to comply with DIN EN 61326-1 when operated on a PCIe compliant main board housed in a properly shielded enclosure. When operated in a closed standard compliant enclosure the device does not pose any hazards as defined by DIN EN 61010-1.

Radiated emissions, noise immunity, and safety highly depend on the quality of the enclosure. It is the responsibility of the system integrator to ensure that the assembled system is compliant to applicable standards of the country that the system is operated in, especially regarding user safety and electromagnetic interference.

When handling the board, adequate measures must be taken to protect the circuits against electrostatic discharge (ESD). All power supplied to the system must be turned off before installing the board.

Environmental Conditions for Storage

The board shall be stored between operation under the following conditions:

Symbol	Parameter	Min	Typ.	Max	Units
T _{store}	ambient temperature	–30		60	°C
RH _{store}	relative humidity at 31°C noncondensing	10		70	%

Environmental Conditions for Operation

Symbol	Parameter	Min	Typ.	Max	Units
T _{oper}	ambient temperature	5		40	°C
RH _{oper}	relative humidity at 31°C	20		75	%

Attention

Do not connect any DC-coupled inputs to a channel while the *TiGer* of that channel is configured as an output. Doing so could permanently damage the TimeTagger4 and the external hardware.

Cooling

The TimeTagger4 in its base configuration has passive cooling that requires a certain amount of airflow. If the case design can't provide enough airflow to the board, a slot cooler like Zalman ZM-SC100 can be placed next to the board. Active cooling is also available as an option for the board.

Recycling

cronologic is registered with the “Stiftung Elektro-Altgeräte Register” as a manufacturer of electronic systems with Registration ID DE 77895909. The TimeTagger4 belongs to category 6, “Kleine Geräte der Informations- und Telekommunikationstechnik für die ausschließliche Nutzung in anderen als privaten Haushalten.” Devices sold before 2018 belong to category 9, “Überwachungs und Kontrollinstrumente für ausschließlich gewerbliche Nutzung.” The last owner of a TimeTagger4 must recycle it, treat the board in compliance with §11 and §12 of the German ElektroG, or return it to the manufacturer’s address listed under [Manufacturer](#).

7 Revision History

Firmware

Gen 1

0.1219 – 2024-07-15

- PCIe interface optimizations
- Fixed first-level FIFO overflow
- Fixed rare bug that causes PC freezes

0.1187 – 2023-05-25

- Extended standard range of measurement to 24 bits
- Fixed wrong polarity flag bug
- Internal optimizations

0.1132 – 2021-12-09

- Fixed possible register read issues

0.1118 – 2021-06-23

- Fixed register write issues

0.983 – 2019-03-15

- Internal optimizations

0.971 – 2019-02-19

- Hit sorting and packet generation issues fixed

Gen 2

0.24178 – 2024-06-26

- Fixed bug related to the PCIe interface
- Internal optimizations

0.24052 – 2024-02-21

- Activated AER capabilities
- Improved alignment for TT4-10G
- Added jitter filter for TT4-10G
- Added option to discard empty packets before they reach the host buffer

0.23180 – 2023-06-29

- Initial release

Driver & Applications

1.12.1 – 2025-07-17

- Added support for kernel mode driver 1.5.2
- Improved error handling

1.10.7 – 2024-07-25

- Support for TimeTagger4-10G (incl. calibration tool)
- Include baseline calibration
- Reduced supported range of dc_offset by 100 mV

1.9.0 – 2023-07-10

- Added quantization to timetagger4_param_info structure
- Code refactorization

1.8.3 – 2023-06-07

- Minor bug fixes
- Code refactorization

1.8.2 – 2023-05-17

- Added bounds and checks for various parameters

1.8.1 – 2023-05-09

- Renamed autotrigger mode to continuous mode

1.8.0 – 2023-05-05

- Added configurable input delay

1.7.0 – 2023-04-18

- Board Revision 7 support
- TimeTagger4: added autotrigger mode

1.4.5 – 2022-10-17

- kernel driver v1.4.2 for xTDC4 only (fixes crash on Windows for Thunderbolt hot-plug)

1.4.4 – 2022-06-27

- kernel driver v1.4.1

1.4.2 – 2021-07-28

- Firmware updated
- ReadoutGUI added/updated
- User guide example added/updated

1.4.1 – 2019-11-11

- x64 32 mode issues fixed

1.4.0 – 2019-06-04

- Improved Windows 10 support

1.3.0 – 2019-01-23

- Added Windows 10 support

User Guide

2.0.5 – 2026-04-29

- Improved *timetagger4_channel.start* and *stop* documentation for *Continuous Mode*
- Improved *timetagger4_start_capture()* documentation
- Updated User Guide layout

2.0.4 – 2026-03-31

- Updated *Status LEDs of the PCIe boards*

2.0.3 – 2026-03-05

- Added *Cable Delay Test*
- Fixed typos

2.0.2 – 2026-01-07

- Updated *Figure 1.3*
- Fixes in *TDC Measurement Characteristics for Gen 2*

2.0.1 – 2025-11-04

- Various clarifications and corrections in API
- Updated *Calibration of the 10G variant*
- Updated introductory text
- Updated *Figure 1.1*
- Added board dimensions
- Clarifications in *Technical Data*
- Added 10 MHz external clock SMA connector to *Figure 1.3*

2.0.0 – 2025-08-01

- Release of the HTML User Guide
- Separated xHPTDC8, xTDC4, and TimeTagger4 User Guides

1.10.7 – 2025-07-12

- xTDC4 and TimeTagger4: Documented driver release 1.12.1

1.10.6 – 2025-06-10

- xTDC4 and TimeTagger4: Updated *Figure 3.2*

- Removed references to J6 connector

1.10.5 – 2025-04-23

- TimeTagger4 and xTDC4: Fixed code examples
- Documented minimal pulse height

1.10.4 – 2025-03-20

- TimeTagger4: Updated Table 8.2.2

1.10.3 – 2025-03-18

- xHPTDC8: Documented Firmware 2.1225 and Driver 1.5.0

1.10.2 – 2024-12-17

- xHPTDC8: Updated Figure 2.3
- xTDC4: Fixed formatting

1.10.1 – 2024-09-19

- xHPTDC8: Fixes and additions to introduction
- xHPTDC8 and TimeTagger4: Documented max. readout rate for single channels
- Updated Figure 2.4

1.10.0 – 2024-08-14

- TimeTagger4: Renamed 10G calibration tool
- Added Section “Memory Layout”

1.9.4 – 2024-07-30

- Updated driver and firmware revision lists
- xHPTDC8: Updated user guide example.cpp

1.9.3 – 2024-07-16

- xHPTDC8: Fix driver revision list

1.9.2 – 2024-07-09

- xHPTDC8: Added LED documentation
- TimeTagger4 and xTDC4: Add overview figure of TBT and PCIe variant
- Fixed grammar

1.9.1 – 2024-07-02

- xHPTDC8: Updated firmware list

1.9.0 – 2024-06-27

- Added new driver revision
- TimeTagger4 and xTDC4: Added TBT variant
- TimeTagger4 and xTDC4: Added ordering information
- TimeTagger4 and xTDC4: Updated supported range for dc_offset

1.8.17 – 2024-06-20

- xTDC4: Fixed API documentation

1.8.16 – 2024-06-20

- TimeTagger4: Added documentation for 10G calibration tool
- xTDC4 and TimeTagger4: Added LED documentation
- xHPTDC8: Fixed default values for zero_channel
- Clarifications for TiGer block indices

1.8.15 – 2024-05-08

- Fixed auto_trigger formula
- Updated oscillator characteristics
- xHPTDC8: Fixed mistakes in API
- xHPTDC8: Updated Code Examples

1.8.14 – 2024-03-27

- Updated API
- Updated information on power consumption
- xHPTDC8: Extended chapter on gating

1.8.13 – 2024-01-18

- xHPTDC8: Updated cover
- TimeTagger4: Updated feature list

1.8.12 – 2024-01-10

- xHPTDC8: Updated driver revision history

1.8.11 – 2023-11-29

- Reformatting
- Added latency between signal and Tiger output to Section 3.5
- TimeTagger4: Updated table in Section 8.1.2
- TimeTagger4: Clarifications in Features-list
- TimeTagger4: Added ignore_empty_packets API documentation
- xHPTDC8: Added default values for manager and configuration structs
- xHPTDC8: Fixed number of boards that can be synchronized from 8 to 6

1.8.10 – 2023-07-28

- Changed extended range values to 0.429s and 2.147s, respectively.
- API clarifications.

1.8.9 – 2023-07-10

- TimeTagger4 User Guide rework

1.8.8 – 2023-03-15

- New TimeTagger4 variants -1.25G to -10G added

1.8.7 – 2022-11-24

- Firmware revision notes updated

1.8.6 – 2022-11-23

- Spelling and grammar corrections
- New example source code for xHPTDC8

1.8.5 – 2021-12-17

- Clarifications related to TimeTagger4 configuration.

1.8.4 – 2021-12-08

- Updated grouping structure in xHPTDC8 API

1.8.3 – 2021-07-28

- Updated firmware revision history

1.8.2 – 2021-04-23

- Added software trigger and _SYNC trigger sources for xHPTDC8
- Corrected 3.3V power requirement for xHPTDC8
- Changed types with fixed bit width to stdint.h for xHPTDC8
- Added user flash functions for xHPTDC8

1.8.1 – 2021-04-09

- Many corrections and updates to the xHPTDC8 API

1.8.0 – 2021-03-22

- Added xHPTDC8 User Guide

1.7.0 – 2021-02-04

- Combined User Guide for -1G and -2G
- Added characteristics for INL, DNL and Time Base
- Reordered sections for clarity
- Error corrections for rollovers, binsize and range
- Added figure 3.2 (TiGer matrix)
- Corrected board revision

1.3.0 – 2019-06-05

- API clarifications

8 Erratum

We found undesired behavior for Gen 1 devices of the TimeTagger4.

If there are three or more edges close together (within 6.6 ns) and the user did only enable rising or falling edges but not both, some edges are reported with the wrong polarity.

If your configuration enables both edges all output data is correct. If you only need one type of edge (rising or falling) there are three simple workarounds:

- a) Update the Firmware of your Gen 1 device to svn1187 or later.
- b) Enable both edges.
All output words will be correct and your software can ignore all data that doesn't have the desired polarity.
- c) Enable only the desired edge polarity.
Ignore the polarity flag in the output data. You can trust that only edges with the desired polarity are output, even if the flag in the data word states the wrong polarity.